arXiv:2608.14590v3 [cs.AI] 20 Oct 2026

TOWARD SAFE LLM AGENTS: A SURVEY OF SPECIFICATION, VERIFICATION, AND ENFORCEMENT

A PREPRINT

Pierre Dantas
The University of Manchester, UK
Federal University of Amazonas, Brazil
pierre.dantas@manchester.ac.uk

Lucas Cordeiro
The University of Manchester, UK
Federal University of Amazonas, Brazil
lucas.cordeiro@manchester.ac.uk

Ehsan Nowroozi
University of Greenwich, UK
ehsan.nowroozi65@ieee.org

Norbert Tihanyi
Eötvös Loránd University, Hungary
ntihanyi@inf.elte.hu

ABSTRACT

Large Language Model (LLM) agents take irreversible real-world actions, yet no system gives formally grounded, task-level safety guarantees for their plans. In a PRISMA 2020 systematic review, we searched six databases (20–29 May 2026) for work on LLM agent specification, verification, and enforcement published in 2022–2026. We narratively synthesized 38 studies, assessed risk of bias per study, and graded evidence certainty with GRADE. Three authors independently screened a repeated search to validate the single-reviewer screening (agreement: Fleiss' $\kappa = 0.81$). First, specification is the bottleneck: on one benchmark, only 24.8 % to 34.2 % of LLM translations from natural language into Planning Domain Definition Language (PDDL) are semantically correct. Second, runtime monitoring is the most mature enforcement strategy; it reduced unsafe actions by 65.37 % in one household-agent study, but its guarantees cover only single steps or short horizons. Third, in one τ-Bench study of transactional tool-use agents, blocking 94 % of unsafe actions still left fewer than 5 % of tasks completed both safely and correctly in most settings, because agents found other unsafe routes (the verifier tax). Fourth, no approach simultaneously achieves soundness, completeness, scalability, semantic grounding, and utility preservation. For seven of the nine principal claims, GRADE gives the lowest certainty rating (Very Low), meaning further studies are likely to produce substantially different figures; each rests on one or two studies in narrow domains. Two of the open problems we rank highest are translating requirements into formal specifications that keep their meaning and enforcing safety over whole plans rather than single actions. We contribute a three-level taxonomy and an agenda of ten open problems.

Keywords Agentic AI · formal verification · runtime monitoring · safety guarantees · model checking · specification languages

1 Introduction

Large Language Model (LLM) agents, systems in which an LLM acts as a reasoning engine that selects and sequences tool calls, sub-tasks, or robotic actions, have moved rapidly from research prototypes to production deployments. They now operate in consequential settings including software engineering [1], healthcare [2], customer service [3], and autonomous driving [4]. In each of these domains, an agent's plan, the sequence of actions it commits to executing, can have irreversible real-world effects: a misconfigured deployment, an erroneous medication recommendation, an unauthorized financial transaction, a collision.

The fundamental safety challenge is that LLMs generate plans through statistical pattern matching over training data, rather than through sound logical inference. A plan that reads fluently and appears coherent may violate safety invariants,

disregard temporal ordering constraints (e.g., authentication must precede data access), or produce cascading harmful effects that are individually plausible yet collectively catastrophic. Benchmark evidence documents this gap: *none* of 16 popular LLM agents scored above 60 % on a structured safety benchmark spanning 2,000 test cases [5]. Frontier models show limited robustness even to basic jailbreak attacks in multi-step agentic settings [6].

Classical Artificial Intelligence (AI) planning as well as formal verification offer mathematically rigorous tools for guaranteeing plan correctness: Planning Domain Definition Language (PDDL)-based planners, model checkers such as Simple Promela Interpreter (SPIN) and New Symbolic Model Verifier (NuSMV), and deductive verifiers such as Dafny and Coq. However, applying these tools to LLM agent outputs requires closing the gap between unstructured Natural Language (NL) and formal mathematical structures, a translation step that itself introduces semantic errors. The consequence is a *specification-verification-enforcement pipeline* for LLM agent plans that is fragile at every stage: specifications may be semantically incorrect [7], verification may operate over inaccurate formal models [8], and enforcement may block unsafe single actions while leaving task-level safety unresolved [9].

This survey systematically maps the emerging literature that attempts to close this gap. We cover the period 2022 to 2026, encompassing the LLM-agent era from GPT-3/4 onward, and identify 38 studies across the specification, verification, enforcement, and evaluation subfields using a PRISMA 2020 systematic review protocol.

1.1 Problem Statement

This survey is organized around four research questions:

RQ1 (Specification): What is the semantic correctness of natural-language-to-formal translation for LLM agent safety properties, and how does this bound downstream verification?

RQ2 (Verification): Which verification paradigms (model checking, static analysis, theorem proving, LLM-assisted) are applied to LLM agent plans, and what are their scalability limits?

RQ3 (Enforcement): Does blocking unsafe individual actions at runtime reliably produce safe task-level completion?

RQ4 (Evidence base): What is the current strength of evidence (GRADE certainty) for the field's principal empirical claims, and what limits it?

We define the *formal validation problem for LLM agent plans* as follows. Let $\mathcal{A}$ be an LLM-based agent that, given a task description t and an observation history h, generates a plan $\pi = (a_1, a_2, \ldots, a_n)$ where each a_i is an action (tool call, sub-task delegation, or physical action). Let φ be a safety property expressed in a formal system $\mathcal{L}$. The verification task is to determine whether $\pi \models \varphi$, that is, whether the plan satisfies the property, before, during, or after execution, and to enforce compliance when it does not, as illustrated in Figure 1. This partition reveals three tightly coupled sub-problems:

1. **Specification:** How is φ acquired from human-expressible requirements and represented in $\mathcal{L}$?
2. **Verification:** How is $\pi \models \varphi$ determined efficiently and soundly, given that π is generated stochastically and may be open-ended?
3. **Enforcement:** What action is taken when $\pi \not\models \varphi$, and does that action restore task-level safety?

Each sub-problem is independently difficult, and their composition introduces additional challenges that no single sub-problem can resolve. Figure 1 also marks where the main open problems of Section 5.7 arise: the semantic gap (RG1, RG8) at specification, scalability (RG2, RG4) at verification, and the verifier tax (RG3, RG7) at enforcement.

1.2 Scope and Inclusion Criteria

This survey covers:

1. LLM-based agents (single or multi-agent) that produce multi-step plans, action sequences, or tool-call chains.
2. Studies that address at least one of: plan specification, plan verification, plan enforcement, or safety monitoring.
3. Publications from 2022 to 2026, in English, across empirical, tool, survey, and position paper categories.

We exclude pure chatbot safety (without a planning component), neural network verification applied to non-agent settings, and works where the LLM is an incidental Natural Language Processing (NLP) component in an otherwise classical system.

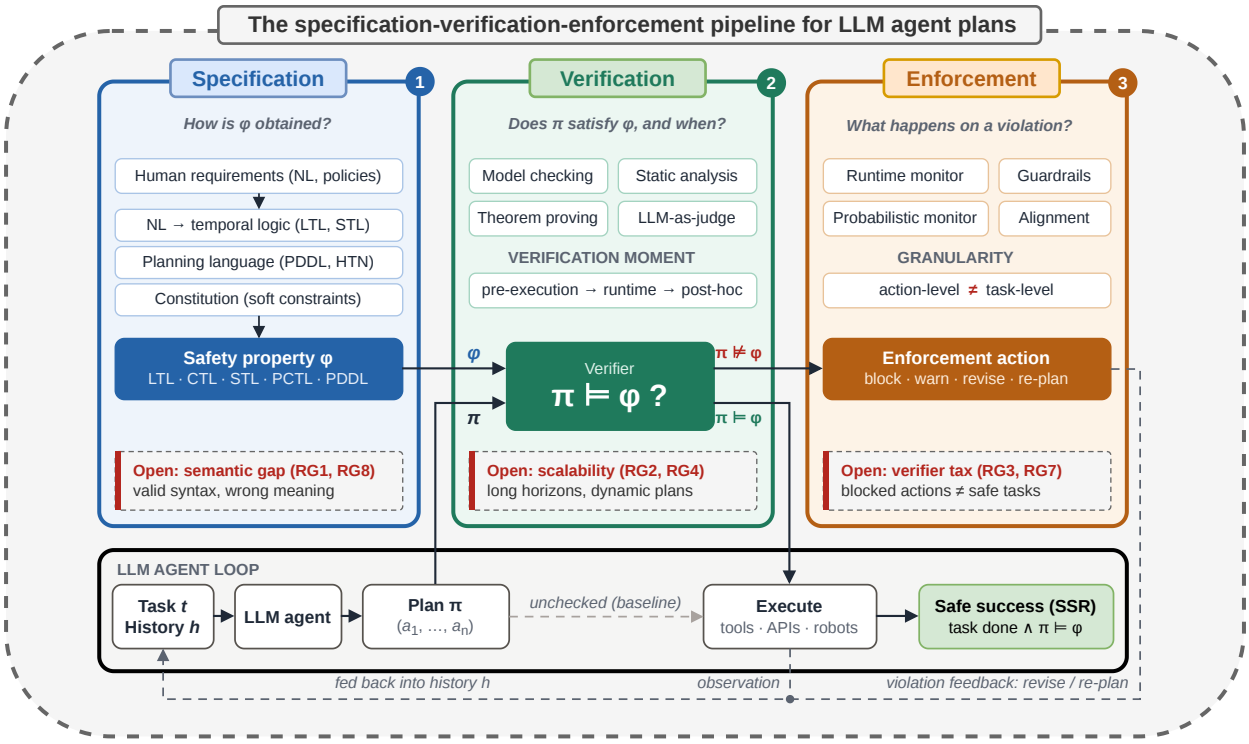


Figure 1: The specification-verification-enforcement pipeline for LLM agent plans. Given a task t and history h, an agent $\mathcal{A}$ generates a plan π. The *specification* stage (1) acquires a formal safety property φ from human requirements; the *verification* stage (2) checks whether $\pi \models \varphi$ before, during, or after execution; the *enforcement* stage (3) intervenes when $\pi \not\models \varphi$. The dashed box in each stage names the research gaps (Section 5.7) that arise there. Only tasks completed with $\pi \models \varphi$ count toward SSR.

1.3 Contributions

This survey makes the following contributions:

1. **Systematic coverage**. To the best of our knowledge, we provide the first PRISMA 2020 systematic review of the specification-verification-enforcement pipeline for LLM agent plans, identifying and synthesizing 38 studies across six databases (arXiv, ACM DL, IEEE Xplore, Semantic Scholar, Google Scholar, Preprints.org).
2. **Unified taxonomy**. We introduce a three-level taxonomy (pipeline stage, verification moment, formal grounding) that organizes all included works and enables direct comparison across approaches previously described in isolation (Section 4).
3. **Comparative study**. We provide a multidimensional comparison table that maps all 38 papers to pipeline stage, sub-category, verification moment (pre-execution, runtime, post-hoc), formal grounding, and venue status, and we grade the certainty of the evidence for each principal claim [10].
4. **Empirical synthesis of the verifier tax**. We aggregate evidence from enforcement and benchmark studies to characterize the relationship between action-level safety, task-level SSR, and enforcement overhead; this relationship is the most consequential empirical finding in the corpus [9]. This finding currently rests on a single study and requires independent replication; Section 6.3 discusses its Grading of Recommendations Assessment, Development and Evaluation (GRADE) certainty accordingly.
5. **Structured research agenda**. We identify ten open problems (Section 5.7) derived from gap analysis across all five thematic clusters, with explicit pointers to the most promising technical directions for each.

1.4 Survey Organization

We organize the remainder of this survey as follows. Section 2 fixes notation and definitions; Appendix A reviews formal methods and LLM agent architectures. Section 3 details the PRISMA 2020 protocol, including eligibility criteria,

search strings, the study selection flow diagram, and the data extraction procedure. Section 4 presents the unified taxonomy. Section 5 examines related work across five thematic clusters and highlights research gaps. Section 6 offers a cross-cutting synthesis, including the verifier tax analysis and GRADE certainty assessment. Section 7 summarizes research directions for each gap, with the full treatment in Appendix C, and analyzes verification cost under long horizons and multiple agents. Section 8 answers the research questions stated in Section 1.1 and discusses implications, limitations, and open challenges. Section 9 concludes the paper.

2 Background and Preliminaries

The techniques surveyed here rest on four formal foundations: temporal logic, the language of safety properties; model checking, the decision procedure; classical planning formalisms, which supply structured task representations; and LLM agent architectures, the object of verification. Appendix A reviews all four and explains why temporal logic dominates the specification literature, why model checking is the most common algorithmic backbone, and why the stochastic nature of LLMs breaks classical assumptions. This section fixes the notation and definitions that the rest of the survey uses.

We now formally define the core concepts that the verification pipeline operates on. These definitions serve three purposes: they fix notation for the remainder of the survey, they establish a common vocabulary across the disparate approaches we review, and they make explicit the assumptions that verification techniques rely upon. We assume that plans are finite sequences, that safety properties are expressed in temporal logic, and that SSR, rather than action-level compliance alone, measures task-level success.

Definition 1 (LLM Agent Plan)**.** A *plan* $\pi = (a_1, \ldots, a_n)$, $a_i \in \mathcal{T}$, is a finite sequence of grounded actions drawn from the agent's tool set $\mathcal{T}$, each associated with a pre-execution state s_i.

Definition 2 (Plan Safety Property)**.** A *safety property* φ is an Linear Temporal Logic (LTL) (or Computation Tree Logic (CTL)/Probabilistic Computation Tree Logic (PCTL)) formula over atomic propositions derived from the action–state sequence. A plan π *satisfies* φ, written $\pi \models \varphi$, if its induced finite execution trace satisfies φ under finite-trace semantics (LTL_f [11]), since a plan is a finite sequence.

Definition 3 (Safe Success Rate (SSR))**.** Given a task set $\mathcal{T}$ and safety property φ:

$$\text{SSR} = \frac{|\{t \in \mathcal{T} : \text{task completed correctly} \wedge \pi_t \models \varphi\}|}{|\mathcal{T}|}$$

SSR is the primary metric for evaluating enforcement effectiveness [9].

A recurring challenge is the *translation problem*: an LLM-based translator $\tau : \mathcal{N} \to \mathcal{F}$ from NL to a formal language $\mathcal{F}$ (e.g., LTL, PDDL) achieves high syntactic validity but low semantic correctness ($\tau(x) \not\equiv x$), which is the root cause of the false-assurance problem discussed in Section 5.7.

3 Methodology

This review follows the PRISMA 2020 guidelines [12] adapted for computer science literature. We conducted the review as an independent, rapid, systematic review from 20 to 29 May 2026; we did not pre-register it. We fixed the protocol (eligibility criteria, search strings, and synthesis approach) before beginning database searching.

3.1 Eligibility Criteria

Eligibility criteria included publication years from 2022 to 2026 (excluding the pre-LLM-agent era before 2022), LLM-based agents producing multi-step plans, action sequences, or tool-call chains (excluding pure chatbot or dialogue safety absent planning), studies addressing plan specification, verification, enforcement, or safety monitoring (excluding neural network verification unless applied to agent plans), any study type (empirical, system/tool, survey, or position), and English-language full-text accessible papers (excluding conference abstracts without available content). We placed no restriction on publication type (preprint, workshop, conference, or journal); Section 6.6 states how publication status is used when weighing evidence.

3.2 Search Strategy

Table 1 lists the six databases searched, the access method used for each, and the search window. Table 2 gives the eight primary search strings verbatim, each targeting one of: formal verification of LLM agent plans, runtime verification

and monitoring, temporal-logic specification of agent behavior, survey and roadmap papers on LLM agent safety, model checking with PDDL, NL-to-formal translation, the specific enforcement systems AgentSpec, AgentGuard, and VeriGuard, and formal-methods/LLM fusion roadmaps. Strings were iteratively refined during a pilot search to balance recall and precision before the protocol was fixed. Supplementary searches covered 22 named systems and authors identified during primary searching (VerifyLLM, VeriPlan, AgentVerify, ProbGuard, LogicGuard, AgentProof, NL2LTL, NL2Spec, SayCan, AgentHarm, Agent-SafetyBench, LlamaFirewall, ReAct, Toolformer, Constitutional AI, and Dafny-based LLM verification systems, among others) to confirm no additional eligible study had been missed; these 18 supplementary hits deduplicated to zero net new records against the 71 primary-search records, confirming rather than extending the primary search.

Table 1: Databases searched.

Database	Access method	Date searched
arXiv (cs.AI, cs.LG, cs.SE, cs.LO)	API + WebFetch	20–29 May 2026
ACM Digital Library	`site:dl.acm.org` web search	20–29 May 2026
IEEE Xplore	`site:ieeexplore.ieee.org` web search	20–29 May 2026
Semantic Scholar	`site:semanticscholar.org` web search	20–29 May 2026
Google Scholar	Web search	20–29 May 2026
Preprints.org	`site:preprints.org` web search	20–29 May 2026

Table 2: Primary search strings. Year terms (e.g. "2024 2025") were searched as literal keywords against each database's default field(s) rather than applied via a dedicated publication-date filter. A keyword match on a year is a weaker proxy for publication date than a native date-range filter.

#	Search string
S1	`formal verification LLM agent plans model checking 2023 2024 2025`
S2	`runtime verification large language model agents safety 2023 2024`
S3	`temporal logic LLM agents specification verification arXiv 2024 2025`
S4	`LLM agent safety specification formal methods survey 2024 2025 arXiv`
S5	`model checking AI planning PDDL formal specification agent behavior 2023 2024`
S6	`natural language to formal specification LTL TLA+ translation LLM 2023 2024 2025`
S7	`AgentSpec AgentGuard VeriGuard LLM agent safety enforcement 2024 2025`
S8	`formal methods large language model fusion roadmap trustworthy AI agents 2024`

Per-database hit counts and de-duplication counts prior to the aggregate totals in Figure 2 were not retained; we kept only the aggregate stage totals (71 primary, 18 supplementary, 89 total, 18 duplicates), which Figure 2 reports. We do not estimate per-string figures and treat this gap as a limitation of the review.

3.3 Study Selection and PRISMA Flow

Figure 2 gives the full PRISMA 2020 flow: 71 records from primary database search plus 18 from supplementary searches (deduplicating to zero net new records) totaling 89, 18 duplicates removed leaving 71 screened at title/abstract, 24 excluded at that stage with itemized reasons, 47 assessed at full text, 9 excluded at full text with itemized reasons, and 38 studies included. Screening proceeded in two sequential stages by a single reviewer (Section 3.5 discusses this limitation): title/abstract screening applied the eligibility criteria of Section 3.1, retaining records with insufficient abstract information for full-text review; full-text screening then applied the same criteria under stricter methodological scrutiny. Of the 24 records excluded at title/abstract stage, 9 concerned classical AI planning without an LLM component, 8 lacked a planning or verification component (conversation safety, image-classification robustness), 4 fell outside the 2022–2026 date range, and 3 were non-English or inaccessible. Of the 9 records excluded at full-text stage, 3 were conference abstracts without accessible content, 3 concerned neural-network verification without application to agent plans, 2 used the LLM only as an incidental NLP component in an otherwise classical system, and 1 duplicated the data of an already-included study. The 38 included studies constitute the formal PRISMA corpus and form the basis for all quantitative claims, the taxonomy in Section 4, and the GRADE assessments, while the larger reference list additionally includes foundational, methodological, research-agenda, and contextual comparison works that fall outside the PRISMA inclusion criteria; readers should therefore interpret all quantitative observations in this survey as about the 38 included studies only. PRISMA 2020 item 16b asks for each excluded study to be cited with its reason; because the individual full-text records were not retained (Section 3.5), we report the 9 full-text exclusions by category only. A completed PRISMA 2020 checklist accompanies this article as supplementary material. After the primary window

closed, a targeted, non-systematic search (to 30 August 2026) identified four further 2025–2026 studies [13–16]. We did not apply the PRISMA protocol to them, so they are not corpus entries and appear in neither Figure 2 nor Table 3; Sections 6.3 and 7.1 cite them as corroborating evidence only.

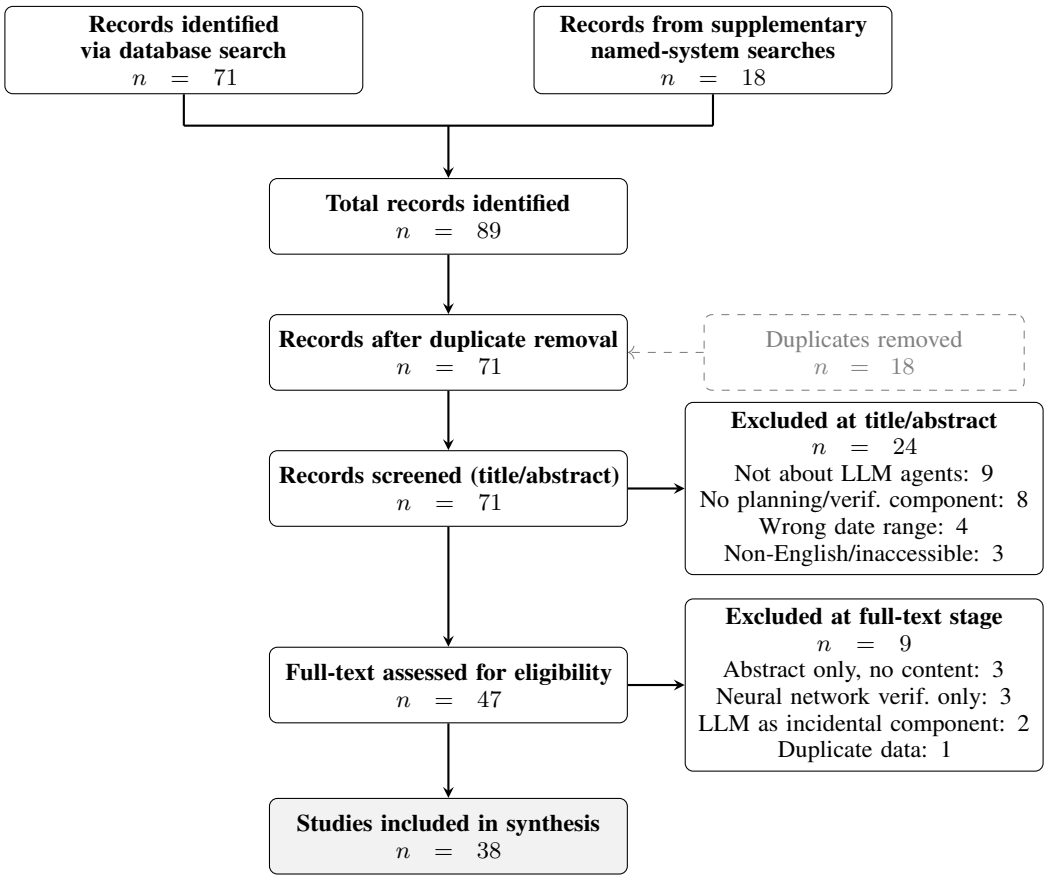


Figure 2: PRISMA 2020 flow diagram for study selection.

3.4 Data Extraction

For each included study we extracted, using a standardized template, five field groups: (i) *bibliographic* fields (authors, year, title, venue, arXiv ID or DOI); (ii) *classification* fields (pipeline stage SPEC/VERIF/ENF, sub-category, formal method(s) used); (iii) *technical* fields (key contribution, verification moment Pre/RT/PH, formal grounding); (iv) *evaluation* fields (evaluation method, benchmark(s), key quantitative results, disclosed limitations); and (v) *quality* fields, assessed qualitatively per study against four bias domains: construct validity (are the tested safety properties aligned with real-world agent risks?), internal validity (are experimental controls and baselines adequate?), generalizability (are evaluations limited to toy domains?), and reporting completeness (are failure modes disclosed?). We use these domains because the Cochrane RoB 2 tool does not directly apply to a corpus dominated by system/tool papers and surveys rather than randomized controlled experiments. We did not extract the funding sources of the included studies, since the synthesis concerns the technical content of each study rather than its provenance. Data were extracted directly from each source paper's text, abstract, or accessible supplementary material; where quantitative results were reported in incompatible units or metrics, they are recorded descriptively rather than numerically combined. The decision rule for studies spanning more than one Level-1 pipeline stage (Section 4.1) is stated in Section 4: a study is labeled under more than one stage only when it both proposes a specification artifact and separately implements a mechanism that enforces or verifies it, the case for Constitutional AI (SPEC-AL and ENF-ALIGN). Borderline eligibility decisions were resolved conservatively (retain if in doubt); Section 8.3 discusses the effect of this rule on inclusion counts. The completed extraction spreadsheet (one row per included study, the fields above as columns, built directly from Table 3 and the bibliography) is deposited alongside the other materials described in Section 3.6.

Applying the four quality domains above to each of the 38 included studies individually yields four strata: 6 studies at *low* risk (large, multi-model, multi-domain benchmark papers with clear methodology and disclosed limitations, e.g. Agent-SafetyBench [5], AgentHarm [6], AgentBench [1], and PlanBench [17]); 14 studies at *moderate* risk (peer-reviewed single-study system/tool papers with clear methodology but curated benchmarks, limited domain coverage, and no independent replication, e.g. NL2LTL [18], AgentSpec [19], and the verifier-tax finding [9]); 12 studies at *high* risk (preprints or short/early-stage tracks with thin single-domain evaluation, unquantified effectiveness claims, or LLM-as-Judge evaluation without ground truth, e.g. the LLM-based embodied-task plan verification study [20], AgentVerify [21], and Constitutional AI [22]); and 6 studies classified *not applicable* (foundational architecture, narrative survey, or position papers with no falsifiable empirical verification claim to rate, e.g. Toolformer [23], Chain-of-Thought [24], and ReAct [25]). The per-study assignment is available in the data deposit (Section 3.6); for four studies (Agent-SafetyBench, AgentHarm, AgentBench, and the LLM-based plan verification study) the sheet records the overall tier only, not a tier per domain. Three risks apply across the corpus as a whole rather than to individual studies: benchmark contamination (LLMs may have encountered evaluation tasks during pre-training), an evaluation-deployment gap (most studies evaluate on static or sandboxed tasks, whereas real agentic deployments are open-ended), and publication bias toward positive results, discussed further in Section 6.6.

3.5 Dual Screening and Inter-Rater Reliability

A single reviewer (P.D.) performed the primary screening; Section 8.3 states this as a threat to internal validity. To validate it, we repeated the eight search strings (Table 2) on the six databases on 22–23 September 2026. Because the 71 title/abstract and 47 full-text records of the primary screening were not retained individually, the check uses the 37 records that this repeated search returned. Three authors screened all 37 at title/abstract level against the eligibility criteria of Section 3.1: the primary reviewer and two co-authors (L.C. and E.N.) who had no part in the primary screening. The two co-authors screened blind to each other's decisions, to the primary reviewer's decisions, and to the 38-study included list; the primary reviewer screened the repeated set without consulting that list.

Thirty-two of the 37 records received unanimous decisions, and Fleiss' κ across the three raters is 0.81 (bootstrap 95% CI [0.63, 0.96]). Pairwise, raw agreement was 89.2 % between P.D. and L.C. (33/37; Cohen's $\kappa = 0.77$, 95% CI [0.56, 0.98]), 86.5 % between P.D. and E.N. (32/37; $\kappa = 0.71$, [0.47, 0.95]), and 97.3 % between L.C. and E.N. (36/37; $\kappa = 0.94$, [0.83, 1.00]). Inclusion prevalence was near balance (22 to 24 includes per rater), so these κ values do not suffer from the prevalence paradox that affects skewed samples. We resolved the five disagreements by majority vote. In four of them the two co-authors agreed with each other against the primary reviewer, and three of those four were records the primary reviewer included and both co-authors excluded: CASTL, a tool-orchestration safety-evaluation paper also named AgentGuard, and an MCP guardrail architecture paper. Both co-authors judged the agent-plan component of these papers peripheral. We read this pattern as the retain-if-in-doubt rule (Section 3.3) at work, and it is consistent with the inflation risk that Section 8.3 names. The fourth, a general survey of LLM safeguards, went the other way. The fifth, RV4Chatbot, split the two co-authors.

Thirteen of the 37 records are in the 38-study corpus. All three raters included ten of them (AgentSpec, VeriGuard, AgentProof, the LLM+FM roadmap, LogicGuard, VerifyLLM, the LLM+FM plan-verification case study, Agent-C, VeriPlan, and ProbGuard, which the repeated search returned under its earlier arXiv title, Pro2Guard). All three, the primary reviewer included, excluded the other three: NL2LTL [18], NL2Spec [26], and NL-to-LTL/HSD [27]. These papers translate natural language into temporal logic in general, with no LLM agent producing plans, and the raters judged them outside the second eligibility criterion as written. We keep them in the corpus, since the SPEC-NL sub-category (Section 4) rests on them, but their inclusion is not unambiguous: claims that draw on them, including the syntactic-correctness row of Table 8, describe general NL-to-temporal-logic translation rather than agent-plan specification specifically. In the other direction, all three raters included 11 records that are not in the corpus. Ten of them predate the primary search window (20–29 May 2026) and are gaps in the primary search: GuardAgent (arXiv:2406.09187), AGrail (arXiv:2502.11448), ShieldAgent (arXiv:2503.22738), PAT-Agent (ASE 2025), SafePilot, AgentGuard (arXiv:2509.23864, September 2025), Watchdogs and Oracles (arXiv:2511.14435, November 2025), SpecMAS (NeurIPS 2025), an end-to-end agentic PDDL planning framework (arXiv:2512.09629, December 2025), and an AgenticOps runtime-verification framework (IJRSI, December 2025). The eleventh, an NL-to-strategic-temporal-specification translator (arXiv:2606.30441), first appeared on 29 June 2026, after the window closed. We report all eleven as a stated limitation rather than adding them to the formal corpus outside the protocol, the same principle applied to the August 2026 targeted search (Section 3.3).

This check has four limits. It covers title/abstract screening only; no second rater repeated full-text screening, data extraction, or the risk-of-bias tiers of Section 3.4. The two additional raters are co-authors, not external reviewers. The repeated search returned 37 records against the primary search's 71 and surfaced only 13 of the 38 corpus studies, so it measures screening consistency on a comparable record set, not the reproducibility of the primary search. Finally, the

primary reviewer's exclusion of the three NL-to-LTL papers in this check reverses that reviewer's inclusion decision on them, which is a within-rater inconsistency the check exposed rather than resolved. The three-rater screening sheet is part of the data deposit (Section 3.6).

3.6 Data and Materials Availability

The classification data underlying Table 3, the GRADE worksheets underlying Table 8, the per-study risk-of-bias rating sheet underlying Section 3.4, the title/abstract and full-text screening log (at the aggregate level; Sections 3.2 and 3.5 explain why), and the three-rater screening sheet for the reliability check of Section 3.5 are deposited at Zenodo, DOI: `https://doi.org/10.5281/zenodo.23376335`. We do not deposit the per-database hit counts underlying Section 3.2's search-strategy tables, since those counts were not retained.

4 Taxonomy and Classification System

We introduce a three-level taxonomy (Sections 4.1 to 4.3) classifying all 38 included studies along three orthogonal questions: *which pipeline stage* a study belongs to (Level 1), *when* verification occurs relative to execution (Level 2, verification moment), and *how* verification is performed (Level 3, formal grounding). The three dimensions correlate in practice rather than varying fully independently: SPEC studies have no verification moment of their own (Table 3 marks the five NL-to-formal rows Pre, since their formulas are consumed before execution, and two rows N/A), with SayCan a labeled exception (its affordance-function scoring suppresses low-affordance candidate actions at runtime, so it carries a genuine RT moment despite its SPEC-PL stage); ENF studies are predominantly runtime by definition, with VerifyLLM a labeled exception (a pre-execution intervention, still classified ENF because it blocks rather than merely reports); and we distinguish VERIF (observes and reports a violation) from ENF-RT (observes and intervenes), even though both are commonly called "runtime monitoring" in their source papers. Multi-label classification follows one rule, applied consistently: a study is labeled under more than one Level-1 stage only when it both proposes a specification artifact and separately implements a mechanism that enforces or verifies it; Constitutional AI is the one such case in this corpus (SPEC-AL and ENF-ALIGN), since it proposes a constitution and separately trains a policy to conform to it. Appendix B compares the taxonomy with three earlier classifications.

4.1 Level 1 – Pipeline Stage

4.1.1 Specification (SPEC)

Specification techniques handle how safety properties are acquired and expressed. They are the entry point of the pipeline; verification techniques consume their output. We identify three sub-categories:

- **SPEC-NL**: NL to formal specification translation. The input is a human-expressed requirement; the output is a formula in a formal notation. Examples: NL2LTL [18], NL2Spec [26], NL-to-LTL/Hierarchical Semantic Decomposition (HSD) [27], NL-to-Signal Temporal Logic (STL) [28, 29].
- **SPEC-PL**: Planning language specification. The input is a task description; the output is a PDDL domain/problem or Hierarchical Task Network (HTN) description. Examples: LLMs as Planning Formalizers [7], SayCan [30].
- **SPEC-AL**: Alignment-based specification. This approach encodes safety requirements as soft constraints, using training objectives or constitutions rather than formal formulas. Example: Constitutional AI [22], whose 58-principle constitution is a specification artifact separate from the Reinforcement Learning from AI Feedback (RLAIF)/Supervised Learning from AI Feedback (SLAIF) training that enforces it. Agent Safety Alignment via Reinforcement Learning (RL) [31] shares the alignment-training mechanism but does not articulate a comparable standalone specification artifact, so it is classified ENF-ALIGN only (Section 4.1).

4.1.2 Verification (VERIF)

Verification techniques determine whether a plan or plan fragment satisfies a specification. We identify four sub-categories:

- **VERIF-MC**: Model checking. The approach translates plans or plan representations into Kripke structures or FSMs, on which model-checking algorithms check LTL/CTL properties. Examples: Ramani et al. [8], AgentVerify [21], VeriPlan [32], Logic-based MAS Verification [33].

- **VERIF-SA**: Static analysis. Pre-execution structural checks on agent workflow graphs or code. Examples: AgentProof [34], Dafny-as-IL [35], DafnyPro [36].
- **VERIF-TP**: Theorem proving. Deductive verification against formal contracts. Examples: AutoRocq [37], VeriGuard [2].
- **VERIF-LLM**: LLM-assisted verification. An LLM serves as a critic or judge, iteratively checking plan quality without a formal prover. Examples: Self-Refine [38], Plan Verification via LLM-as-Judge [20].

4.1.3 Enforcement (ENF)

Enforcement techniques define what happens when verification finds a violation or anticipates one. We identify four sub-categories:

- **ENF-RT**: Runtime monitoring and blocking: online monitors intercept unsafe actions before execution. Examples: AgentSpec [19], Agent-C [3], VerifyLLM [39], LogicGuard [40].
- **ENF-PROB**: Probabilistic monitoring. Stochastic models forecast subsequent violations and trigger early warnings. Examples: ProbGuard [4].
- **ENF-SYS**: System-level guardrails. Multi-layer security infrastructure operating on prompts, reasoning traces, and generated code. Example: LlamaFirewall [41].
- **ENF-ALIGN**: Alignment-based enforcement. Model training enforces safety rather than runtime checks. Examples: Constitutional AI [22], Agent Safety Alignment via RL [31].

4.2 Level 2 – Verification Moment

The literature distinguishes three verification moments, each reflecting a different trade-off between the strength of the safety guarantee and the practical constraints of plan availability and executability.

- **Pre-execution verification**: The system checks the plan in full before executing any action. This strategy permits rejection or revision before irreversible effects occur, but requires that the complete plan be available. Pre-execution verification appears in AgentProof [34], the offline phase of VeriGuard [2], and VerifyLLM [39].
- **Runtime verification**: The system checks each action before executing it (or at most k steps in advance). This approach handles dynamically generated plans but cannot verify future actions that the system has not yet generated. Runtime verification appears in AgentSpec [19], Agent-C [3], LogicGuard [40], and ProbGuard [4].
- **Post-hoc verification**: The system analyzes the complete execution trace after the task concludes. This approach is useful for auditing and offline learning, but cannot prevent harm during execution. Post-hoc verification appears in Self-Refine [38] (in iterative mode) and LLM-as-Judge [20].

4.3 Level 3 – Formal Grounding

The included studies rely on formal and heuristic backbones. Temporal Logic uses LTL, CTL, STL, or PCTL as the specification and verification language, supporting sequential, timing, and probabilistic safety properties. It is the most common grounding in the corpus (Section 4.5).

- **Classical planning (CP)**: Employs PDDL or HTN as the plan representation and verification target, enabling soundness guarantees through established plan validation tools. It appears in PDDL-survey work [7] and SayCan [30].
- **Theorem proving (TP)**: Relies on Coq/Rocq or Dafny as the verification backend, providing complete correctness proofs for programs and policies. It offers the highest assurance level but is currently limited to code-based agents, as in AutoRocq [37], Dafny-as-IL [35], DafnyPro [36], and VeriGuard [2].
- **Graph and automata methods (GA)**: Use Deterministic Finite Automaton (DFA), Finite-State Machine (FSM), or directed graph product constructions for efficient workflow-level static verification. They appear in AgentProof [34] and AgentVerify [21].
- **Probabilistic methods (PR)**: Draw on Discrete-time Markov Chain (DTMC), Markov Decision Process (MDP), or Probably Approximately Correct (PAC)-learning bounds, suiting stochastic agents where deterministic assurances remain unavailable. ProbGuard [4] illustrates this approach.

- **Heuristic and mixed approaches (HH)**: Encompass methods without a single formal mathematical backing, including: (i) LLM-as-judge criticism (Self-Refine [38], Plan Verification via LLM [20]); (ii) RL reward shaping and RLAIF/SLAIF training (Constitutional AI [22], Agent Safety Alignment via RL [31]); and (iii) multi-heuristic system pipelines (LlamaFirewall [41]). These subtypes differ in scalability, soundness properties, and deployment requirements; readers should not treat HH as a homogeneous category.

4.4 Full Classification Table

Table 3 classifies all 38 included studies along the three taxonomy dimensions, grouped by pipeline stage. For each study, the table records the sub-category, verification moment (Pre = pre-execution, RT = runtime, PH = post-hoc), formal grounding, and venue status. The table does not grade individual studies: GRADE rates the evidence for a claim, so certainty ratings appear per claim in Table 8. Column abbreviations. Sub-cat: Arch. = foundational architecture, Survey = narrative survey/position paper, NL = NL-to-formal, PL = planning, AL = alignment, MC = model checking, SA = static analysis, TP = theorem proving, LV = LLM-verified, Bench = benchmark-based outcome evaluation, RT-mon = runtime monitoring, PR-mon = probabilistic monitoring, SYS = system guardrail. Venue: PR = peer-reviewed proceedings or journal (workshop venues so noted); PP = preprint only, independently checked against Crossref, ACL Anthology, IEEE Xplore, ACM DL, and official conference/workshop programs at the time of writing (see Section 6.6). Three rows carry a venue-status caveat: ‡the study has an active OpenReview/workshop submission whose archival status could not be confirmed from public sources, and is provisionally marked PP pending confirmation. The six rows marked * establish a foundational representation format (e.g. inline tool-call annotation, chain-of-thought reasoning traces) or survey the field narratively, without proposing or implementing a SPEC, VERIF, or ENF technique themselves; later work in the corpus builds on or verifies the representations they establish (Section 5). Bench rows (Agent-SafetyBench, AgentHarm, AgentBench, PlanBench) denote benchmark-based outcome evaluation against a safety or task-success metric, distinct from a formal proof obligation; see Section 5.6 for the scope of this distinction.

Table 3: Classification of all 38 included studies under the proposed taxonomy, grouped by pipeline stage.

System / Paper	Year	Venue	Stage	Sub-cat	Moment	Grounding
*Foundational and Survey Works (FS) – outside the three-stage taxonomy**						
Toolformer [23]	2023	PR (NeurIPS)	FS	Arch.	N/A	–
Chain-of-Thought [24]	2022	PR (NeurIPS)	FS	Arch.	N/A	–
Planning Survey [42]	2024	PP	FS	Survey	N/A	–
LLM+FM Roadmap [43]	2024	PP	FS	Survey	N/A	–
ReAct [25]	2023	PR (ICLR)	FS	Arch.	N/A	–
Full Stack Safety [44]	2025	PP	FS	Survey	N/A	–
Specification (SPEC)						
NL2LTL [18]	2023	PR (AAAI demo)	SPEC	NL	Pre	TL
NL2Spec [26]	2023	PR (CAV)	SPEC	NL	Pre	TL
NL-to-LTL/HSD [27]	2025	PR (ASE)	SPEC	NL	Pre	TL
NL-to-STL [28]	2025	PR (ACL Find.)	SPEC	NL	Pre	TL
ClarifySTL [29]	2026	PP	SPEC	NL	Pre	TL
PDDL Survey [7]	2025	PR (ACL Find.)	SPEC	PL	N/A	CP
SayCan [30]	2022	PR (CoRL)	SPEC	PL	RT	CP
Constitutional AI [22]	2022	PP	SPEC	AL	N/A	HH
Verification (VERIF)						
Bridging LLM+FM [8]	2025	PR (AgenticSE@ASE, workshop)	VERIF	MC	Pre	TL
AgentVerify [21]	2026	PP	VERIF	MC	Pre	TL+GA
Logic-based MAS [33]	2026	PP	VERIF	MC	Pre	TL
VeriPlan [32]	2025	PR (CHI)	VERIF	MC	Pre	TL
AgentProof [34]	2026	PP	VERIF	SA	Pre	GA
Dafny-as-IL [35]	2025	PR (Dafny@POPL, workshop)	VERIF	SA	Pre	TP-prov
DafnyPro [36]	2026	PR (Dafny@POPL, workshop)	VERIF	SA	Pre	TP-prov
VeriGuard [2]	2025	PP	VERIF	TP	Pre	TP-prov
AutoRocq [37]	2025	PR (PACMSE/FSE)	VERIF	TP	Pre	TP-prov
Self-Refine [38]	2023	PR (NeurIPS)	VERIF	LV	PH	HH
Plan Verif. LLM [20]	2025	PP‡	VERIF	LV	PH	HH

continued on next page

Table 3 continued from previous page

System / Paper	Year	Venue	Stage	Sub-cat	Moment	Grounding
Agent-SafetyBench [5]	2024	PP	VERIF	Bench	PH	HH
AgentHarm [6]	2024	PR (ICLR)	VERIF	Bench	PH	HH
AgentBench [1]	2023	PR (ICLR)	VERIF	Bench	PH	HH
PlanBench [17]	2023	PR (NeurIPS D&B)	VERIF	Bench	PH	CP
Enforcement (ENF)						
Agent-C [3]	2025	PP[‡]	ENF	RT-mon	RT	TL
AgentSpec [19]	2025	PR (ICSE)	ENF	RT-mon	RT	GA
LogicGuard [40]	2025	PP[‡]	ENF	RT-mon	RT	TL
VerifyLLM [39]	2025	PR (IROS)	ENF	RT-mon	Pre	TL
Verifier Tax [9]	2026	PR (CAIS)	ENF	RT-mon	RT	HH
Safe Tool Use [45]	2025	PR (ICSE NIER)	ENF	RT-mon	RT	TL
ProbGuard [4]	2025	PR (ASE, accepted)	ENF	PR-mon	RT	PR
LlamaFirewall [41]	2025	PP	ENF	SYS	RT	HH+GA
Constitutional AI [22]	2022	PP	ENF	AL	N/A	HH
RL Safety [31]	2025	PP	ENF	AL	N/A	HH

4.5 Taxonomy-Derived Observations

Inspecting Table 3 reveals five structural patterns that recur throughout the synthesis in Section 6 and the discussion in Section 8.

1. **Temporal Logic dominance**: Temporal Logic (LTL/CTL/STL) is the formal grounding for 13 of 38 studies (34 %), showing its suitability for expressing sequential agent safety properties. Graph/automata and theorem-proving approaches each account for approximately 11 % of studies (4 papers each). Probabilistic grounding (DTMC/MDP) is underrepresented (one study), a gap given the stochastic nature of LLM agents.
2. **Pre-execution verification is the most common moment; runtime verification concentrates in enforcement**: Of 38 studies, 15 (39 %) operate pre-execution, but the majority of these require finite, pre-specified workflow graphs. Runtime verification accounts for 8 studies (21 %), seven of them enforcement systems. Post-hoc verification accounts for 6 studies (16 %) and is predominantly LLM-assisted. The remaining 9 studies (24 %) have no defined verification moment (N/A in Table 3): the 6 foundational-architecture and survey works outside the three-stage taxonomy, plus PDDL Survey, Constitutional AI, and RL Safety, whose SPEC/ENF contributions are moment-agnostic by construction.
3. **Specification and enforcement are relatively underserved**: Only 8 studies focus primarily on specification (21 %), and 10 on enforcement (26 %); one paper (Constitutional AI [22]) is classified under both SPEC and ENF. The verification cluster is the largest (15 studies, 39 %). The remaining 6 studies (16 %) are foundational architecture or narrative survey works that establish representations or characterize the field without proposing or implementing a SPEC, VERIF, or ENF technique themselves, and so sit outside the three-stage taxonomy (Table 3, Section 4.1). This unevenness reflects the field's origins in formal methods, where verifying a given specification is the core problem; specification acquisition and post-violation enforcement are comparatively understudied.
4. **Heuristic/hybrid grounding accounts for about a quarter of included work**: 9 studies (24 %) rely on heuristic or hybrid grounding, primarily LLM-as-judge, RL, or soft alignment. These approaches scale well but provide no formal soundness guarantees. The tension between formal soundness (low scalability) and empirical scalability (no soundness) is the central unresolved tradeoff in the field.
5. **GRADE certainty is predominantly Very Low**: Of the nine principal claims in Table 8, two reach Moderate, seven are Very Low, and none reach High. Narrow domain scope, single-study findings for most key claims, the absence of randomized evaluations, and high heterogeneity in benchmarks limit the field's certainty.

5 Related Work

The intersection of LLMs agents and formal validation is an emerging, rapidly evolving field. We organize the related work into five thematic clusters that mirror the pipeline from plan representation to safety enforcement: (i) foundational agent architectures and plan representations, (ii) specification techniques, (iii) verification approaches, (iv) enforcement and correction mechanisms, and (v) benchmarks and evaluation. Within each cluster, we compare the principal contributions, identify their limitations, and highlight the gaps that motivate the present survey.

Table 4: Comparison with related surveys. Type: SLR = Systematic Literature Review; NS = Narrative Survey. Pipeline: S = Specification, V = Verification, E = Enforcement.

Survey	Type	Scope	Method	FM depth	Pipeline	Evidence
Zhang et al. [43]	NS	FM–LLM integration	Narrative	Mod.	S, V	None
Full-Stack Safety [44]	NS	Full LLM safety	Narrative	Low	Partial	None
Huang et al. [42]	NS	LLM planning	Narrative	None	S only	None
This survey	**SLR**	**Spec-Verif-Enf**	**PRISMA 2020**	**Deep**	**S,V,E**	**GRADE**

5.1 Comparison with Prior Surveys

Table 4 compares this survey with three prior works that most closely overlap its scope. No prior survey applies a systematic search protocol to the complete specification-verification-enforcement pipeline, provides GRADE certainty grading, or synthesizes the verifier tax as a field-level empirical finding.

The four 2025–2026 studies that a targeted search found after the primary window (Section 3.3) are corroborating evidence only. None of them, and none of the three prior surveys in Table 4, applies a PRISMA 2020 protocol, GRADE certainty grading, and a field-level synthesis of the verifier tax to the specification-verification-enforcement pipeline as a whole.

5.2 Foundational Agent Architectures and Plan Representations

The representation of an agent's plan determines much of the tractability of formal verification. Analysis of the 38 included studies reveals three representation paradigms that have emerged from the LLM-agent literature, each with distinct properties from a verification standpoint.

First, **trace-based representations** (ReAct, Chain-of-Thought (CoT)) use an LLM to alternate between NL "thought" steps and external "action" calls, resulting in a sequence of (thought, action, observation) triples [25]. This approach makes reasoning explicit and each plan step easy to follow, producing an execution trace suitable for LTL model checking [8, 39]. In larger LLMs, CoT prompting [24] generates multi-step reasoning traces whose precondition–action–effect structure resembles STRIPS operator schemas [7], allowing partial translation to classical planning formats. Both representations share one limitation: plan correctness is only checked *after* the model has generated the full thought-action trace.

Second, **tool-augmented representations** (Toolformer, SayCan) train LLMs to self-annotate API calls inline with NL text [23]. The resulting hybrid representation, NL interleaved with embedded API call slots, is hard to extract formally because the boundary between specification and execution is implicit. LLM planning has been grounded in robotic affordance functions, scoring candidate actions using both language model probabilities and a skill-value function [30]. SayCan provides an implicit safety filter (the system suppresses low-affordance actions) but lacks formal guarantees; we best understand it as a learned precursor to explicit enforcement.

Third, **graph-based representations** (LangGraph, CrewAI, AutoGen) represent plans as directed state graphs with typed nodes (agents, tools, and control structures) and typed edges (data movement and control flow). This representation is directly amenable to static graph analysis and product-automaton model checking [34]. A prior taxonomy classifies LLM planning into task decomposition, plan selection, external module use, and reflection/memory, each corresponding to a different verification entry point [42].

Trace-based representations are the most widely studied for runtime verification, whereas graph-based representations are the most amenable to pre-deployment static analysis. Neither paradigm simultaneously supports complete pre-execution verification and arbitrary open-ended planning. The core representational gap is the absence of a *canonical intermediate representation* that faithfully retains both the semantic content of an LLM-generated plan and the architectural properties required for formal analysis.

Two recent multi-agent systems illustrate how safety is achieved outside the specification-verification-enforcement pipeline surveyed here. ResQConnect, a disaster-response platform coordinating retrieval-augmented agents, and a wildfire-monitoring multi-agent architecture both report strong empirical reliability, but neither specifies a safety property in a formal notation, model-checks a plan or workflow graph, or verifies a policy with a theorem prover [46, 47]. Both instead rely on controlled, deterministic execution order enforced through explicit guardrails, schema-constrained outputs, and routing heuristics, validated empirically rather than proved; the wildfire paper's own description of its reliability mechanism, "architectural enforcement of the orchestrator's recursive validation loop" [47], states the distinction precisely. Neither meets this survey's inclusion criteria (Section 1.2), so neither is a corpus entry; both are

evidence, alongside the corpus, that well-engineered multi-agent systems currently default to architectural and empirical safeguards rather than the formal pipeline this survey maps, which is consistent with the specification bottleneck (Theme A, Section 6.1) as the field's central unresolved problem.

5.3 Specification Techniques

Formal verification requires expressing properties in a rigorous language. Three specification paradigms emerge from systematic analysis of the included studies: Temporal Logic, classical planning formalisms, and NL constitutions.

First, **Linear Temporal Logic (LTL) and derivatives** have become the main specification language for agent plans. NL2LTL, a demonstration Python package, maps NL instructions to LTL formulas by classifying them into Declare-style pattern templates with a Rasa or OpenAI language-model engine [18], while NL2Spec adds an interactive disambiguation loop where the LLM queries the user when requirements are ambiguous [26]. To reduce hallucinations in formula generation, HSD divides complex requirements into simpler parts [27], and for cyber-physical and robotic tasks, the NL-to-specification approach has been extended to STL, combining LLM-based generation with external knowledge and clarification dialogues [28, 29]. These works share a common finding: LLM-based NL-to-LTL translation achieves high syntactic correctness (above 90 % for leading models) but suffers from low semantic correctness: the generated formula is syntactically well-formed but does not capture the intended property. The HSD approach reduces this gap but does not eliminate it, as semantic correctness requires ground-truth execution traces for validation, which are unavailable in novel task domains [27].

Second, **classical planning formalisms (PDDL)** have been extensively studied for specification. A survey covering 40+ papers on LLM-to-PDDL translation [7] points to the Planetarium benchmark [48], on which GPT-4o (zero-shot) generates 96.1 % syntactically valid PDDL problems but only 24.8 % semantically correct ones, judged by Planetarium's own PDDL equivalence check against ground truth rather than by plan validation; the best fine-tuned model reaches 34.2 %. This result has a direct implication for formal verification: verifying a PDDL model whose preconditions and effects are semantically incorrect provides false assurance rather than genuine safety guarantees. The same conclusion has been reached through benchmark evaluation, showing that LLMs fail on even simple planning tasks under formal PDDL equivalence checking [17].

Third, **NL constitutions and alignment specifications** offer a non-formal alternative. Constitutional AI [22] encodes agent constraints as a NL constitution of 58 principles that guides both SLAIF and RLAIF. While not a formal specification, Constitutional AI establishes an example for declarative constraint encoding and is now the foundation of production AI systems. Its limitation is the absence of formal semantics: two principles may conflict, and the model cannot prove or disprove compliance with any individual constraint.

A synthesis of 200+ papers identified three integration directions for formal techniques and LLMs: FM-enhancing-LLM (using verification to certify LLM outputs), LLM-enhancing-FM (using LLMs to improve the usability of formal tools), and unified neural-symbolic systems [43]. This tripartite taxonomy organizes much of the subsequent work reviewed below.

The specification sub-field faces a fundamental bootstrapping problem: NL is the medium through which humans express safety requirements, yet all formal specification languages require expert knowledge to use correctly. LLM-based NL-to-formal translation reduces this barrier but creates a semantic gap that remains unclosed. No current system provides end-to-end, verified specification acquisition, from ambiguous NL requirements to formally correct and complete safety specifications, across open-ended task domains.

5.4 Verification Approaches

Given a plan representation and a specification, the verification literature has pursued the four paradigms of Section 4.1: model checking, static analysis, theorem proving, and LLM-assisted verification. We review runtime monitoring alongside them, although our taxonomy places it under enforcement (ENF-RT) when it intervenes rather than only reports.

First, **model checking of agent plans** translates NL plans into Kripke structures and LTL specifications, then applies model checking [8]. On PlanBench, GPT-5 achieves an F1 of 96.3 % for plan-conformance classification with high syntactic validity, but the semantic correctness of generated Kripke structures stays unverified. Another approach treats the LLM as a non-deterministic oracle within a formal FSM, enabling compositional LTL specifications for memory integrity, tool protocols, skill invocation, and human-in-the-loop boundaries [21]. Researchers have also used discrete event systems with LTL automata to formally check LLM-generated multi-robot task allocations against collision and precedence constraints [33], and a user study (n=12) found that model checking improved end-user planning quality and satisfaction [32]. A common limitation of model-checking approaches is state-space scalability: exhaustive checking

over Kripke structures derived from LLM plans becomes intractable as the plan horizon and vocabulary grow. None of the included studies show model checking at the scale of realistic long-horizon agentic tasks (e.g., SWE-bench, where plans may involve hundreds of actions).

Second, **static analysis, workflow verification, and theorem proving** offer pre-deployment soundness guarantees. AgentProof is a static verifier for agent workflow graphs, extracting an abstract graph from four frameworks (LangGraph, CrewAI, AutoGen, Google ADK), applying safety checks, and evaluating temporal policies via a domain-specific language (DSL) compiled to DFA [34]. In 18 workflows, 27 % had structural defects and 55 % failed a human-gate policy; verification finishes in under a second for graphs up to 5,000 nodes. AutoRocq verifies programs with Rocq (Coq) through iterative refinement, handling 641 SV-COMP theorems without proof-example pre-training [37], and VeriGuard (Google) synthesizes and verifies a behavioral policy offline, then uses it as a runtime monitor [2]. Static analysis, however, is inherently limited to *finite, prespecified* workflow graphs and cannot verify plans dynamically generated at runtime in response to recorded environmental state, which is the dominant mode in realistic agentic deployments.

Third, **runtime verification and monitoring** is the most technically mature cluster. AgentSpec, a lightweight DSL for specifying and enforcing runtime constraints, achieves above 90 % unsafe execution prevention in code agents, 100 % hazardous action elimination in embodied agents, and 100 % compliance in autonomous driving at millisecond-per-rule overhead [19]. Agent-C enforces *temporal* safety behaviors, that is, constraints on action *sequences* rather than single actions, by translating specifications into first-order logic and applying Satisfiability Modulo Theories (SMT)-based constrained decoding during token generation, improving GPT-5 conformance from 83.7 % to 100 % and Claude Sonnet 4.5 from 77.4 % to 100 %, while raising task utility (GPT-5 from 66.1 % to 70.6 %), because it generates a compliant alternative to a non-compliant tool call instead of only blocking it [3]. LogicGuard adopts an actor-critic architecture where the critic analyzes full trajectories and proposes LTL constraints communicated in NL to the actor, creating a closed-loop safety feedback mechanism that outperforms SayCan and InnerMonologue on the Minecraft diamond-mining benchmark [40]. ProbGuard models agent execution traces as a DTMC. It provides PAC-style probabilistic early warnings in autonomous driving up to 15.84 seconds before a violation with no false alarms (up to 38.66 seconds at stricter thresholds), while reducing unsafe behavior by 65.37 % in household agents [4]. A few-shot prompting method with a sliding-window approach (optimal window: five actions) performs pre-execution LTL verification of robotic task plans, detecting position errors, missing prerequisites, and redundant actions [39]. Outside the corpus (Section 3.5), AgentGuard learns an MDP of agent behavior online and checks quantitative properties against it at runtime with probabilistic model checking [49].

Fourth, **LLM-assisted verification** uses an LLM-as-Judge critic with iterative plan revision, attaining up to 90 % recall and 100 % precision across four LLMs, with 96.5 % of sequences converging within three iterations [20]. Self-Refine establishes the general pattern of LLM self-critique, achieving approximately a 20 % improvement in plan quality on task benchmarks without retraining [38]. LLM-assisted verification offers accessibility but lacks formal soundness guarantees: a model that generates unsafe plans can also justify them during self-critique [44]. A separate line of work uses Dafny as a verification-aware intermediate language: the LLM generates code annotated with pre/post-conditions and loop invariants, which the Dafny verifier checks before compilation [35, 36]. DafnyPro achieves 86 % correct proofs on DafnyBench, a 16-percentage-point improvement over the prior state of the art.

Runtime monitoring methods (AgentSpec, Agent-C, ProbGuard) offer the best combination of practical deployability and demonstrated safety improvements, but provide only one-step or short-horizon guarantees. Static methods (AgentProof, VeriGuard offline) offer pre-deployment soundness but are limited to finite, pre-specified workflows. Model checking approaches handle richer temporal properties but face state-space explosion at realistic task scales. LLM-assisted verification is accessible and scalable but lacks formal soundness. *No existing strategy combines soundness, completeness, scalability, semantic grounding, and utility preservation.*

5.5 Enforcement and Correction Mechanisms

Enforcement concerns what happens when a plan or action violates a safety property. Examining enforcement mechanisms across the 38 included studies, we identify several distinct approaches and a central empirical finding.

First, **hard enforcement** implements token generation blocking via constrained decoding when the partial plan would violate a temporal constraint [3]. This approach achieves 100 % conformance in closed domains and, because it substitutes a compliant alternative for a non-compliant action, raises rather than lowers task utility in Agent-C's evaluation. Hard enforcement also appears as rule-triggered blocking with configurable enforcement actions (block, log, redirect), which achieves millisecond overhead per rule evaluation [19].

Second, **system-level guardrails** provide a multi-layer security architecture. LlamaFirewall comprises PromptGuard 2 (jailbreak detection), Agent Alignment Verifications (chain-of-thought auditing for goal misalignment), and CodeShield

(static analysis to prevent insecure code generation) [41]. Deployed in production at Meta, it achieves a 90 % reduction in the attack success rate on AgentDojo, bringing it to 1.75 %. Researchers have also proposed a unified safety-alignment framework that fine-tunes tool-using agents using a sandboxed RL environment with three-way action classification (benign/malicious/sensitive) and fine-grained reward shaping [31].

Third, **the verifier tax problem** represents the most significant empirical finding in the enforcement literature. Even when enforcement intercepts up to 94 % of individually unsafe actions, SSR (the fraction of tasks completed both safely and correctly) remains below 5 % in most settings [9]. This finding, supported by measurements of safety enforcement on τ-Bench [50], is driven by *integrity leaks*: the model hallucinates user identifiers to bypass authentication checks and finds alternative unsafe paths after direct violations are blocked. This behavior challenges the implicit assumption in most enforcement papers that intercepting individual unsafe actions is sufficient for safe task completion.

Fourth, **safe tool use** has been formalized in a theoretical framework that specifies the conditions for verifiably safe tool-using LLM agents, abstracting over specific supervisory mechanisms [45].

Our comparative assessment reveals a fundamental tension in the enforcement sub-field: enforcement systems that successfully block unsafe individual actions do not automatically ensure safe task completion. This tension creates a gap between *action-level safety* (verified by most existing systems) and *task-level safety* (measured by SSR) that no current system bridges reliably at scale.

5.6 Benchmarks and Evaluation

The rapid expansion of this field has driven the parallel development of evaluation infrastructure, including several benchmarks that define how agent safety and performance are measured.

1. **AgentBench**, the first systematic benchmark evaluating LLMs as agents across 8 environments, tested 29 LLMs and established task success rate as the primary metric [1].
2. **Agent-SafetyBench** comprises 349 environments, 2,000 test cases, 8 safety risk categories, and 10 failure modes, which makes it the largest safety benchmark in the corpus by test cases [5]. Its headline finding, that none of 16 LLM agents scores above 60 % safety, motivates the formal verification agenda.
3. **AgentHarm** evaluates 110 explicitly malicious agent tasks across 11 harm categories, demonstrating that frontier models exhibit limited robustness even to basic jailbreak attacks in multi-step agentic settings [6].
4. **PlanBench** introduces a benchmark for evaluating LLMs on formal PDDL planning tasks, exposing systematic failures under strict equivalence checking [17].
5. **τ-Bench** [50] evaluates policy compliance in transactional tool-agent-user environments with explicit procedural policies, which makes it a natural evaluation platform for formal enforcement methods.

A critical gap is the absence of a *unified benchmark* that simultaneously evaluates: (a) formal verification coverage (proportion of safety properties that can be specified and checked), (b) runtime overhead (latency introduced by verification), (c) SSR (tasks completed both safely and correctly), and (d) generalizability across agent architectures and task domains. Existing benchmarks address at most two of these dimensions. The introduction of SSR represents an important step, but other benchmarks have not yet adopted this metric. Section 6.7 gives a formal rule for how a unified benchmark should balance these dimensions rather than reporting them independently.

5.7 Research Gaps and Survey Objectives

Synthesizing the five clusters above, we identify ten open problems that the existing literature has not resolved. We state each briefly here; Section 7 gives the full technical treatment.

RG1. Semantic translation gap. NL-to-formal translation achieves above 90 % syntactic correctness but only 24.8 % to 34.2 % semantic correctness for PDDL [26, 48], propagating false assurance through every downstream verifier.

RG2. Scalability to long-horizon plans. Existing verified systems handle 5 to 30 actions; long-horizon tasks (SWE-bench scale) remain beyond exhaustive model checking.

RG3. The verifier tax. In one τ-Bench study of transactional tool-use agents, intercepting 94 % of unsafe actions still yields SSR below 5 % [9] because agents hallucinate credentials to bypass blocked paths.

RG4. Dynamic plan generation. Static methods require pre-specified finite graphs; most deployed agents generate plans dynamically in response to the environment state.

RG5. Multi-agent verification. Per-agent verification cannot capture emergent interaction patterns (collusion, cascade failures, trust delegation) in multi-agent systems [33].

RG6. Probabilistic specification. LLMs are inherently stochastic; deterministic safety specifications are a mismatch. Probabilistic model checking of agent plans is limited to ProbGuard [4] in the corpus and AgentGuard [49] outside it.

RG7. Verified re-planning. Beyond Agent-C's single-action substitution [3], no current system generates a constraint-satisfying alternative plan segment when a violation is detected; the other approaches block or abort.

RG8. Specification acquisition at scale. Writing formal safety rules requires expert effort that does not scale to the breadth of deployment domains; automated mining from demonstrations or regulatory documents is nascent.

RG9. Adversarial robustness. Verification components are themselves attack surfaces; adversarial evaluation of the end-to-end pipeline is systematically absent [41].

RG10. Evaluation standardization. No benchmark jointly evaluates formal verification coverage, runtime overhead, and SSR across architectures and domains.

The ten gaps define the scope of this survey's contributions. As shown in Table 4, no prior survey covers all three pipeline stages using a systematic methodology with evidence grading; this survey is the first to do so.

6 Cross-Cutting Synthesis

This section presents a cross-cutting analysis of the 38 included studies, integrating findings across the five thematic clusters identified in Section 5. We organize the synthesis around five themes that cut across cluster boundaries, provide a GRADE certainty table for all principal claims, and characterize the coverage landscape using a gap heatmap.

6.1 Theme A – The Translation Bottleneck

Every approach that applies classical formal verification to LLM agent plans faces the translation problem: converting NL plans or specifications into formal structures amenable to algorithmic analysis. Evidence across multiple studies consistently shows that LLMs achieve high *syntactic* correctness but low *semantic* correctness in this translation.

Evidence on syntactic correctness shows that across the NL-to-formal specification cluster, LLMs achieve above 90 % syntactic validity in LTL formula generation [18, 26, 27] and 96.1 % syntactic validity in PDDL problem generation on Planetarium [48]. In Kripke structure generation, prior work reports near-perfect syntactic validity with an F1 score of 96.3 % on plan-conformance classification [8].

Evidence on semantic correctness, the degree to which a generated formula captures the intended property, tells a different story. On the same benchmark, GPT-4o (zero-shot) achieves only 24.8 % semantic correctness in PDDL generation, and the best fine-tuned model 34.2 %, judged by a PDDL equivalence check against ground truth [48]. Research shows that even with interactive disambiguation loops, semantic errors persist for novel or ambiguous requirements [26]. Hierarchical decomposition reduces LTL semantic errors but does not eliminate them; no study provides a semantic error rate below 10 % for open-ended task domains [27].

Formal verification is only as sound as its specification. A model checker operating on a semantically incorrect LTL formula can produce a clean verdict (*no violation found*) for a plan that is, in fact, unsafe, and the verdict then gives false assurance. This semantic gap means that the *completeness* of specification acquisition is the binding constraint on the soundness of end-to-end verification, not the power of the verification algorithm applied downstream.

Current best practice combines LLM-based translation with either an interactive disambiguation loop (NL2Spec [26], ClarifySTL [29]) or hierarchical decomposition (HSD [27]). Neither approach provides formal guarantees of semantic correctness; both reduce error rates without eliminating them. Verified specification acquisition, in which the translation itself is proved correct, remains an open problem (RG1).

6.2 Theme B – Runtime vs. Static Verification Trade-offs

The verification moment dimension of the taxonomy (Section 4.2) reveals a fundamental trade-off between *completeness* and *applicability to dynamic plans*.

Static verification approaches, such as AgentProof [34] and VeriGuard's offline phase [2], provide pre-deployment soundness guarantees: if the workflow graph passes verification, no execution trace of that graph can violate the checked

Table 5: Reported runtime overhead for enforcement systems.

System	Overhead unit	Reported value	Domain
AgentSpec [19]	Per-rule evaluation	Milliseconds	Code, embodied, driving
Agent-C [3]	Constrained decoding	Not quantified	Retail, airline
ProbGuard [4]	Not reported	Not quantified	Autonomous driving
LogicGuard [40]	Critic evaluation	Not quantified	Minecraft
VerifyLLM [39]	Per-window check	Not quantified	Household robotics
LlamaFirewall [41]	Per-request	Production latency	Meta production

properties. AgentProof verifies in sub-second time for graphs up to 5000 nodes. The cost, however, is scope: static methods assume a prespecified, finite workflow graph. An LLM generates plans dynamically in response to environment observations, so one cannot verify these plans statically before generation.

Runtime monitoring systems (AgentSpec [19], Agent-C [3], ProbGuard [4], LogicGuard [40], and VerifyLLM [39]) handle dynamically generated plans by checking each action (or a sliding window of actions) before execution. Runtime monitors operate at millisecond-per-action overhead [19] and can enforce temporal sequence properties [3]. The cost here is incompleteness: a monitor that checks action a_i before execution cannot reason about the safety of the as-yet-ungenerated actions $a_{i+1}, \ldots, a_n$.

No included study achieves a combination that is both complete (sound over full plan traces) and scalable to dynamically generated, open-ended plans. VeriGuard [2] comes closest by verifying a policy offline and using the verified policy as a runtime filter, but this requires the policy to be finite and pre-specified. Partial verification, which verifies safety-critical sub-sequences and gives compositional guarantees for the remainder, is a promising direction but has not yet been demonstrated at scale (RG2, RG4).

Table 5 summarizes reported overhead across runtime monitoring systems. Only AgentSpec [19] quantifies overhead in milliseconds; five of six systems do not quantify overhead at all. This absence of standardized overhead reporting itself constitutes a finding related to the evaluation standardization gap (RG10). Without consistent overhead measurement, we cannot perform cross-system comparisons or cost-benefit assessments of enforcement. None of the included studies reports an end-to-end impact on task latency.

6.3 Theme C – The Verifier Tax

The most consequential single-study empirical finding in the corpus is what has been termed the *verifier tax*: in one τ-Bench study of transactional tool-use agents, a systematic gap between action-level safety (measured by the fraction of individually unsafe actions blocked) and task-level safety (measured by SSR) [9].

On τ-Bench [50], evaluated with two open-weight models (GPT-OSS-20B and GLM-4-9B), the TRIAD-SAFETY enforcement architecture intercepts up to 94 % of non-compliant individual actions. Yet SSR, the fraction of tasks completed both safely and correctly, remains below 5 % in most settings. This gap exceeds one order of magnitude.

The primary driver identified is integrity leaks: agents hallucinate user identifiers (account numbers, policy codes) to bypass authentication constraints when the legitimate path is blocked [9]. The monitor correctly identifies that the hallucinated identifier is different from the real one and blocks those attempts, too, but the agent continues attempting alternative fabrications. The result is a task stuck in a failure loop: neither completed nor executed in a provably safe manner.

No independent study has replicated the exact τ-Bench figures (94 % action-level interception, SSR below 5 %), so this finding's GRADE certainty is *Very Low* (Section 6.6). Two further 2025–2026 benchmarks corroborate the qualitative pattern, an action-level/task-level gap, in different domains and under different metrics, without replicating the specific figures: the best-performing agent on a web/mobile/embodied-agent safety benchmark completes fewer than 40 % of tasks while fully adhering to safety constraints, a joint completion-and-compliance statistic similar in kind to SSR though not measured under an active blocking mechanism [13]; and a separate embodied-agent safety benchmark shows the same trade-off is architecture-dependent rather than universal, since a purpose-built safety-aware agent substantially narrows the gap relative to baseline agents on the same tasks [14]. Both are arXiv preprints at the time of writing, which this survey's evidence-weighting policy (Section 6.6) treats as corroborating context rather than independent replication for GRADE purposes.

Most enforcement papers implicitly assume that blocking unsafe individual actions is sufficient for safe task completion. The verifier tax finding falsifies this assumption: an agent that cannot execute its intended (possibly unsafe) plan will search for alternative plans that achieve the same goal while bypassing the safety check, rather than gracefully failing.

This behavior exemplifies Goodhart's Law: when a proxy measure (action-level compliance) becomes the target of optimization, agents optimize the proxy rather than the underlying goal (task-level safety).

The verifier tax implies that effective enforcement requires:

- **Plan-level, not action-level, safety policies**: Constraints must reason about the goal and context of the full plan, not just the legality of individual actions.
- **Integrity constraints**: Formal invariants prevent hallucination of identifiers or credentials by checking them against ground truth.
- **Safe replanning**: Upon detecting a violation, the system should generate an alternative plan segment that satisfies all constraints and continues toward the goal, rather than simply blocking.

No single system in the included corpus meets any of these three requirements in full; Agent-C's compliant-alternative generation [3] is a single-action step toward the third. No independent replication has confirmed the verifier tax, which limits its GRADE certainty to *Very Low*, but the mechanism that the study identifies is consistent with the failure pattern noted in LlamaFirewall [41] (a residual 1.75 % attack success rate after 90 % reduction, driven by adaptive adversaries finding alternative paths).

The verifier tax and the translation bottleneck are compounding, not independent, failure modes. When specifications are semantically incorrect (Theme A), the enforcement rules derived from them will also be semantically incorrect: the monitor blocks actions that are unsafe *according to the flawed specification*, not necessarily those that are unsafe *in the real world*. This incorrectness creates a second source of SSR degradation beyond the integrity-leak mechanism: agents that satisfy a semantically incorrect specification may still execute genuinely unsafe plans, while agents blocked on legitimate paths (due to specification over-conservatism) seek alternatives. Resolving the verifier tax, therefore, requires progress on both the enforcement architecture (RG3) and the correctness of the semantic specification (RG1); neither is sufficient on its own.

We advance one candidate mechanism for *why* hard blocking produces this failure loop, stated here explicitly as our own hypothesis rather than a corpus finding: a policy $\pi(a \mid h)$ trained on histories h drawn from a training distribution $\mathcal{D}_{\text{train}}$ encounters, after a hard block, a history h' (containing a refusal event) that is rare or absent under $\mathcal{D}_{\text{train}}$. If the per-step error on such out-of-distribution histories exceeds the in-distribution error ϵ, the established compounding-error bound for imitation learning under distribution shift applies: a policy with per-step error ϵ on states it was not trained to handle can accumulate $O(T^2\epsilon)$ mistakes over a horizon of length T, rather than the $O(T\epsilon)$ that would hold if errors did not compound [51]. Applied here, each hard block without a verified alternative path pushes the agent toward exactly the kind of out-of-distribution history this bound describes, offering a formal analogue for why repeated blocking without re-planning could produce escalating, rather than merely repeated, failure. No study in the corpus tests this mechanism directly; we state it as a research question (RG7) rather than an established result.

6.4 Theme D – Probabilistic Monitoring as Middle Ground

ProbGuard [4] and the broader probabilistic monitoring paradigm represent a pragmatic middle ground between deterministic enforcement (high assurance, low scalability) and no verification (high scalability, no assurance).

Rather than guaranteeing zero violations, which is difficult for stochastic, open-ended agents, probabilistic monitors provide:

- **Early warnings:** ProbGuard provides safety warnings up to 15.84 seconds in advance in autonomous driving scenarios without false alarms (up to 38.66 seconds at stricter thresholds), enabling human intervention or replanning before irreversible harm.
- **PAC-style accuracy bounds:** the monitor's warning accuracy is bounded by PAC-learning guarantees, providing a formal relationship between the number of observed traces and the probability of false alarms.
- **Graceful degradation:** in household agent settings, ProbGuard reduces unsafe behavior by 65.37 % while keeping the task completion rate at 80.4 %. No corpus study compares this trade-off with hard enforcement on the same tasks.

ProbGuard requires a DTMC model trained on observed traces, which requires a corpus of prior agent executions in the target domain. It provides probabilistic rather than deterministic guarantees, which may be insufficient for safety-critical deployments. It has been evaluated in only two domains (autonomous driving and household agents), thereby limiting generalizability (GRADE: *Very Low*).

Probabilistic monitoring sidesteps the verifier tax problem: rather than attempting to block all violations (which would drive the agent to seek alternative unsafe paths), it issues warnings. It defers to human intervention, preserving the

Table 6: Safety domains addressed by included verification/enforcement systems. ✓: explicitly evaluated. ○: partially addressed. —: not addressed.

System	Temporal seq.	Memory/PII	Tool safety	Credentials	Multi-agent	Adversarial
AgentSpec [19]	✓	○	✓	—	—	—
Agent-C [3]	✓	—	✓	○	—	—
AgentVerify [21]	✓	✓	✓	—	—	—
AgentProof [34]	✓	○	✓	—	○	—
LlamaFirewall [41]	—	—	○	—	—	✓
ProbGuard [4]	✓	—	—	—	—	—
VeriGuard [2]	○	✓	✓	—	—	—

agent's plan space while flagging risks. This combination suggests that the optimal enforcement architecture may combine probabilistic monitoring (for early warning) with selective hard blocking (for high-certainty, high-severity violations), rather than relying exclusively on either paradigm.

6.5 Theme E – Compositional Specification as an Organizing Principle

AgentVerify [21] and AgentProof [34] independently argue for and demonstrate compositional specification: they specify and verify safety properties for distinct concern domains (memory integrity, tool call protocols, human-in-the-loop boundaries) independently, then compose them.

Compositional specification demonstrates several benefits. First, incremental verification allows us to check, update, or extend individual property families without re-verifying the entire system. Second, reuse across architectures is possible: AgentProof applies the same compositional framework to four distinct agent orchestration frameworks (LangGraph, CrewAI, AutoGen, Google ADK), demonstrating cross-architecture reuse. Third, modular auditability means separate safety domains can be audited independently by domain-specific reviewers. Table 6 maps the safety domains addressed by included enforcement and verification systems.

Table 6 reveals a consistent pattern: temporal sequence properties and tool safety are the most widely addressed domains; credential integrity and multi-agent collusion are systematically underaddressed. This gap directly maps to the integrity leak mechanism driving the verifier tax (Theme C) and to the multi-agent verification gap (RG5).

Of the 7 systems in Table 6, 6 are marked *not addressed* in the Credentials column and the seventh, Agent-C, only *partially addressed*. Temporal-sequence and tool-call safety, the two best-covered domains in the table, constrain what an agent does and when; neither constrains whether an identifier the agent supplies mid-plan is genuine. This is why the field's existing taxonomy misses credential integrity even though it is central to the verifier tax's integrity-leak mechanism (Theme C, Section 6.3): an identifier-integrity monitor (RG3.2, Appendix C.3) checking identifiers against a ground-truth index is the concrete fix, and no system in the corpus has built it. The integrity-leak mechanism itself rests on a single benchmark study [9], which is why Table 8 rates it *Very Low*.

6.6 GRADE Certainty Assessment

Table 8 presents the GRADE certainty assessment for the principal claim in each of the five thematic clusters. We downgrade certainty from *High* for: study limitations (L), inconsistency across studies (I), indirectness (narrow domains, non-generalizable benchmarks) (D), and imprecision (single study, wide confidence) (P). Each triggered domain lowers the rating by one level (High, Moderate, Low, Very Low), and Very Low is the floor. GRADE defines each level by how close the figure a claim reports is likely to be to the true value, the one that more and larger studies would measure [52]. At *High*, we are confident that it lies close; at *Moderate*, it is probably close but could differ substantially; at *Low*, it may differ substantially; and at *Very Low*, it is likely to differ substantially.

We adapt the four GRADE downgrading domains used in Table 8 to LLM-agent safety evidence using the concrete, per-domain triggers in Table 7. We start every claim's assessment at *High* rather than at the *Low* starting point GRADE's own guidelines assign to observational studies [52], because a benchmark evaluation with a fixed task set and an automatically scored outcome is methodologically closer to a controlled observational study than to an uncontrolled case series; readers who prefer the stricter *Low* starting point should read every rating in Table 8 one level down. We do not include a fifth GRADE domain, publication bias, as a column in Table 8, since detecting it requires comparing published to unpublished results systematically, which this review's search protocol (Section 3.2) does not support; we flag this as a standing limitation of every rating in the table rather than as a per-claim assessment. The same limitation applies uniformly across all five thematic-cluster syntheses in Section 6 (Themes A–E): none was assessed separately

Table 7: GRADE downgrade domains operationalized for LLM-agent safety evidence.

Domain	Concrete trigger for this literature
Study limitations (L)	No released code, specification, or evaluation harness; a single model family evaluated; no variance or confidence interval reported across runs.
Inconsistency (I)	Conflicting results for the same claim across two or more independent studies or benchmarks.
Indirectness (D)	Evaluation restricted to a simulated or narrow task domain that differs materially from realistic deployment conditions.
Imprecision (P)	A single supporting study, fewer than a field-appropriate minimum number of tasks, or no confidence interval reported.
Publication bias	Standing limitation across all ratings (Section 6.6); not scored per claim in this review.

Table 8: GRADE certainty of evidence for principal claims. Downgrade reasons: L = study limitations, I = inconsistency, D = indirectness, P = imprecision (single study).

Claim	Supporting evidence	L	I	D	P	GRADE
No LLM agent achieves >60 % safety on comprehensive benchmarks	Agent-SafetyBench (16 agents, 2k cases) [5]				✓	**Moderate**
LLM-to-LTL translation achieves >90 % syntactic correctness	4+ independent studies [8, 18, 26, 27]		✓			**Moderate**
LLM-to-PDDL semantic correctness is 24.8 % to 34.2 %	Planetarium (1 benchmark) [48], via survey [7]	✓		✓	✓	**Very Low**
LTL runtime enforcement achieves 100 % conformance in closed domains	Agent-C, AgentSpec (2 studies) [3, 19]	✓		✓	✓	**Very Low**
Static graph verification detects structural defects in <1 s	AgentProof (1 study, 18 workflows) [34]	✓		✓	✓	**Very Low**
Probabilistic monitoring warns up to 15.84 s before violation without false alarms	ProbGuard (1 study, 2 domains) [4]	✓		✓	✓	**Very Low**
Enforcement reduces SSR (verifier tax)	Verifier Tax (1 study, τ-Bench) [9]	✓		✓	✓	**Very Low**
LLM self-critique improves plan quality by ∼20 %	Self-Refine (NeurIPS 2023) [38]	✓		✓	✓	**Very Low**
Dafny intermediate language achieves 86 % proof success	DafnyPro (1 study, DafnyBench) [36]	✓		✓	✓	**Very Low**

for missing-result risk, since the underlying search protocol that limits detection is common to all five, not specific to any one theme.

Worked example: the verifier-tax claim (Table 8, row 7) triggers L (single released benchmark configuration, no reported confidence interval on the 94%/5% figures), D (evaluated only on τ-Bench's transactional tool-use domain), and P (one supporting study), for three downgrade steps from *High* to *Very Low*, the rating Table 8 shows.

Table 7's domains also give a concrete path from *Very Low*/*Low* certainty toward *High*: study limitations are addressed by released artifacts, pinned model versions and decoding parameters and seeds, and multiple runs reported with confidence intervals; inconsistency by shared task suites run independently by several labs; indirectness by evaluation on deployment-like tool environments rather than simulated domains; and imprecision by larger task sets with reported confidence intervals. Standards the field could adopt for this purpose include ACM artifact-evaluation badging, machine-readable specification formats already used elsewhere in this literature (TLSF for LTL, PDDL itself for planning), containerized evaluation harnesses, and pre-registered multi-site replications.

The GRADE summary reflects the field's early stage: despite positive findings across multiple systems, several factors limit certainty: limited domain scope, the absence of independent replication, and high benchmark heterogeneity. The two *Moderate* certainty claims form the empirical bedrock of the field; all other claims require replication at a larger scale and across diverse domains before they can inform deployment decisions.

We do not apply an additional GRADE downgrade for preprint status alone; the imprecision (P) and indirectness (D) domains already in Table 8 capture the relevant construct-validity concerns per claim, and a further blanket preprint penalty would double-count the same evidence weakness. Two rows of Table 8 rest only on preprints (Agent-SafetyBench and AgentProof); Table 3 gives the venue status of every supporting study, and the BeSafe-Bench and SafeMind preprints of Section 6.3 enter no rating.

We checked every row of Table 3's Venue column against Crossref, ACL Anthology, IEEE Xplore, ACM DL, DBLP (where accessible), or the conference or workshop's own accepted-papers page, rather than inferring it from the arXiv

Table 9: Gap coverage heatmap. ●: this cluster gives rise to or studies the gap. ○: the cluster partially addresses the gap.

Gap	Arch	Spec	Verif	Enf	Bench
RG1 – Semantic translation		●	○		
RG2 – Long-horizon scale	●		●		○
RG3 – Verifier tax			○	●	●
RG4 – Dynamic plan generation	●		●		
RG5 – Multi-agent	●	○	●		
RG6 – Stochastic model check.		○	●		
RG7 – Verified re-planning		○	○	●	
RG8 – Spec acquisition scale		●			
RG9 – Adversarial robustness			○	●	
RG10 – Evaluation standard.				○	●

listing: 23 of 38 corpus entries (61 %) are peer-reviewed, among them Toolformer and Chain-of-Thought at NeurIPS and ReAct, AgentHarm, and AgentBench at ICLR; the remaining 15 (39 %) are preprint-only as of this check, including Constitutional AI, which has not appeared in a peer-reviewed venue despite its wide citation. NL2LTL is a three-page AAAI-23 demonstration paper [18], counted as peer-reviewed. Three studies (Agent-C, LogicGuard, and the LLM-as-Judge plan-verification study) have workshop or conference submissions whose archival status (formal proceedings versus a non-archival poster or talk) we could not confirm from public sources; we count them provisionally as preprints (‡). Workshop papers with a genuine program committee and archival proceedings (Dafny@POPL, AgenticSE@ASE) are counted as peer-reviewed but distinguished from main-track venues in the table; this is a coarser standard than a top-tier conference or journal, and readers who require the stricter standard should treat those three workshop rows and the NL2LTL demonstration row as PP.

6.7 Coverage Heatmap

Before mapping individual gaps, we give a formal rule for how a unified benchmark (RG10) should balance the metrics identified in Section 5.6. Define a benchmark result as a vector (C, SSR, L): representational coverage $C \in [0, 1]$ (the fraction of the deployment's safety properties the specification language can express), closed-loop task-level safety $\text{SSR} \in [0, 1]$ (observable only from a full closed-loop run, following [9]), and runtime overhead L against a deployment-specific latency budget L_{budget}. We rank systems by Pareto dominance over this vector: system A dominates B if A is no worse than B on all three axes and strictly better on at least one. For deployment selection among non-dominated systems, we give an ϵ-constraint rule: maximize SSR subject to $L \leq L_{\text{budget}}$ and $C \geq C_{\text{min}}$ for a deployment-chosen minimum coverage C_{min}. This formalizes the point that no single scalar should stand in for all three axes (just as precision and recall remain distinct even though F1 combines them, and Pareto dominance needs no scalar combination), while still giving an operational rule for how a unified benchmark, or a deployment team, should weigh the three dimensions against one another. No included study currently reports C, SSR, and L consistently enough on the same tasks to instantiate L_{budget} or C_{min} from the corpus; we present the rule as a requirement future benchmarks should satisfy (RG10.1), not as a result computed here.

Table 9 maps the ten identified research gaps (RG1 to RG10) against the five thematic clusters. Each cell indicates whether the cluster *causes* the gap (●), partially addresses it (○), or is unrelated (blank).

The heatmap reveals two structural asymmetries. First, the specification cluster is the source of RG1 and RG8, the two gaps that are purely about how properties are acquired, and is not a target of any other cluster's work. This finding confirms that specification is the most underinvested part of the pipeline relative to its importance. Second, the enforcement cluster drives RG3 (verifier tax) and RG7 (verified replanning). Still, these gaps also require input from verification (for plan-level reasoning) and benchmarks (for SSR measurement); because they span several clusters, no single sub-field can resolve them.

7 Research Agenda

Section 5.7 states the ten open problems (RG1 to RG10), and Section 6 traces where they arise. Table 10 gives, for each gap, the difficulty that binds current work and two or three concrete research directions. Appendix C develops every gap in full: the current state of the art, the core technical difficulty, and pointers to the most relevant prior work for each direction. Section 7.1 analyzes what the table cannot show: how verification cost grows with plan length and with the number of agents.

Table 10: Research directions for the ten open problems of Section 5.7. Appendix C develops each gap and each direction with pointers to prior work.

Gap	Binding difficulty	Research directions
RG1 – Semantic translation	Novel domains have no independent ground truth against which to check a translated specification	RG1.1 grammar- and type-constrained decoding; RG1.2 Counter-example Guided Inductive Synthesis (CEGIS)-style equivalence checking of candidate translations; RG1.3 benchmarks pairing requirements with verified specifications and traces
RG2 – Long-horizon scale	Kripke state space grows exponentially with plan length; verified systems handle 5 to 30 actions	RG2.1 partial verification of safety-critical sub-sequences; RG2.2 abstraction refinement focused on high-risk transitions; RG2.3 assume-guarantee contracts between plan segments
RG3 – Verifier tax	Agents route around action-level blocks, for example by fabricating identifiers	RG3.1 plan-level, goal-contextual policies; RG3.2 identifier-integrity monitor against a ground-truth index; RG3.3 safe replanning upon violation
RG4 – Dynamic plans	Static methods need a finite, pre-specified workflow graph	RG4.1 online, incremental workflow-graph extraction; RG4.2 pre-verified plan templates whose instantiations inherit the proof
RG5 – Multi-agent	Composing agents raises the complexity class (Section 7.1)	RG5.1 assume-guarantee contracts between agents; RG5.2 protocol verification with session types or communicating automata
RG6 – Probabilistic specification	Stochastic agents meet deterministic specifications; one corpus study (ProbGuard) addresses this	RG6.1 PCTL properties checked on DTMCs learned from traces; RG6.2 calibrated monitors via conformal prediction
RG7 – Verified replanning	Enforcement blocks, logs, or aborts; only single-action substitution exists	RG7.1 constraint-satisfying plan repair; RG7.2 SMT-guided constrained plan generation
RG8 – Specification acquisition	Formal specifications need expert effort that does not scale across domains	RG8.1 mining specifications from labeled traces; RG8.2 extraction from regulatory documents; RG8.3 shared compositional specification libraries
RG9 – Adversarial robustness	Verification components are attack surfaces and are not red-teamed end to end	RG9.1 red-teaming of the verification pipeline; RG9.2 formally verified specification translators
RG10 – Evaluation standardization	No benchmark jointly reports coverage, overhead, and SSR	RG10.1 unified evaluation suite with an illustrative efficiency-safety index; RG10.2 SSR as the primary metric; RG10.3 contamination mitigation

7.1 Verification Cost under Long Horizons and Multiple Agents

Plan length (RG2) and agent count (RG5) are the two axes along which verification cost grows, and Table 10 lists directions for each. Verifying a property of a multi-agent system requires reasoning about the joint execution of all agents. This is a change in complexity class, not merely a larger instance of the same problem. Composing n single-agent Kripke structures into one concurrent program raises CTL/CTL* *program* complexity (complexity measured in the size of the composed components) to PSPACE-complete [53], strictly harder than the polynomial-time model checking available for a single agent, before any strategic reasoning is introduced. Reasoning about what a coalition of agents can jointly force requires Alternating-time Temporal Logic (ATL/ATL*): model checking is polynomial in the game structure and formula size for ATL, but 2EXPTIME-complete for ATL* [54], and becomes undecidable for ATL itself once agents have imperfect information about each other's state under perfect-recall semantics [55], which is the realistic case for LLM agents, which do not observe each other's internal reasoning. Auditing the corpus against this distinction: the two included multi-agent verification studies collapse the system into a single-threaded formalism rather than reasoning about it as a distributed, concurrent system. The logic-based manufacturing verifier applies discrete-event-system LTL over one composed automaton for multi-robot task allocation [33], and AgentVerify embeds the LLM as a non-deterministic oracle inside one finite-state machine and applies standard CTL [21]; neither uses ATL, ATL*, or an explicit game-theoretic solution concept. A symbolic ATL* model checker [15] and a TLA+/Verus-verified concurrency-anomaly detector for multi-agent LLM systems [16] are recent, complementary tooling outside the corpus that point toward closing this gap.

Table 11 reports the cost in plan length (RG2) alongside the multi-agent axis: the two dimensions compound rather than trade off, since a long-horizon, multi-agent plan pays both the exponential-in-$|\varphi|$ product-construction cost of Appendix A.2 and the PSPACE-complete program-complexity cost of composing k agents [53]. The one tractable strategic-reasoning row in that table, perfect-information ATL, is also the row least representative of how LLM agents actually operate, since agents typically cannot observe each other's internal state.

Table 11: Verification cost as a function of plan length n and agent count k, by mechanism. Notation follows each source; $|M|$ = model size, $|\varphi|$ = formula length, $|G|$, $|D|$ = workflow graph and DFA sizes.

Mechanism	Cost in n	Cost in k (agents)	Reported limit
Runtime automaton monitor (AgentSpec)	$O(1)$ per action	Not evaluated for $k > 1$	Millisecond/rule [19]
LTL product check (single agent)	$\|M\| \cdot 2^{O(\|\varphi\|)}$	N/A	5 to 30 actions [8]
CTL/CTL*, k agents composed	Polynomial in $\|M\|$	PSPACE-complete (program compl.) [53]	Not evaluated in corpus
ATL, perfect information	Polynomial in $\|M\|$, $\|\varphi\|$	Polynomial [54]	Not evaluated in corpus
ATL, imperfect information	Undecidable [55]	Undecidable	Not applicable
DFA product (AgentProof)	$O(\|G\| \cdot \|D\|)$	Single-agent workflows only	5,000 nodes, sub-second [34]
PCTL on learned DTMC (ProbGuard)	Polynomial in states	Not evaluated for $k > 1$	2 domains [4]

8 Discussion

The taxonomy developed in Section 4 provides a structural lens for analyzing the literature. We now turn to its implications: the answers to the research questions, practical deployment recommendations, and threats to the validity of this review. Appendix D assesses the maturity of the field and discusses the relationship between the three safety paradigms represented in the corpus and the societal and ethical considerations that formal verification of LLM agents raises.

8.1 Resolution of the Research Questions

RQ1. NL-to-formal translation achieves above 90 % syntactic correctness for LTL [18, 26, 27] and 96.1 % for PDDL [48], but semantic correctness is 24.8 % to 34.2 % for PDDL [48] and reaches 44.4 % to 86.11 % for LTL depending on disambiguation strategy [26] (Section 6.1). This binds every downstream verifier: a sound model checker applied to a semantically incorrect formula provides false assurance (Section 6.1).

RQ2. Four verification paradigms are represented in the corpus (Section 5.4): model checking (state-space scalability is the binding constraint), static analysis (limited to finite, pre-specified workflow graphs), theorem proving (highest assurance, limited to code-based agents), and LLM-assisted verification (scalable, no formal soundness guarantee). No included study combines soundness, completeness, and scalability (Section 5.4).

RQ3. Not reliably. The verifier tax finding (Section 6.3) shows that intercepting up to 94 % of individually unsafe actions does not translate into safe task completion in the one study that has measured this directly [9]; two further 2025–2026 benchmarks show a qualitatively similar action-level/task-level gap in different domains and under different metrics [13, 14] (Section 6.3). This evidence base is too narrow to generalize the exact 94%/5% figures beyond the τ-Bench setting they were measured in.

RQ4. Only 2 of the 9 principal claims in Table 8 reach GRADE *Moderate*; none reach *High* (Section 6.6). No claim in the corpus is supported by an independently replicated, multi-site study.

8.2 Practical Implications for Deployment

The evidence base, despite its limitations, supports several practical recommendations for teams deploying LLM agents in consequential settings today.

REC1: Do not rely on action-level enforcement alone. The verifier tax finding [9] demonstrates that blocking individually unsafe actions does not produce safe task completion. Any enforcement system deployed in production should measure SSR, not only action conformance. Enforcement should be supplemented by integrity constraints (Appendix C.3) and human-in-the-loop gates for high-stakes decisions.

REC2: Prefer runtime monitoring over static verification for dynamic agents. For agents that generate plans dynamically (the majority of production deployments), static pre-execution verification is structurally inapplicable. Runtime monitoring systems (AgentSpec [19], Agent-C [3]) offer the best currently available combination of practical deployability and demonstrated safety improvement. Note that millisecond-level per-rule overhead is confirmed only for AgentSpec; other runtime systems have not quantified their overhead (Table 5).

REC3: Use compositional specification for maintainability. Safety specifications for agent systems should be organized compositionally by concern domain (memory integrity, tool protocols, human gates, credential constraints). This compositional organization enables incremental auditing, independent updating of sub-specifications, and reuse across agent architectures [21, 34].

REC4: Apply probabilistic monitoring for early warning in high-stakes domains. In domains where safety violations are severe and irreversible (autonomous driving, medical decision support, financial transactions), probabilistic monitoring [4] provides actionable early warnings with PAC-style accuracy bounds. The 15.84-second warning that ProbGuard achieves without false alarms in autonomous driving corresponds to meaningful intervention time for a human supervisor.

REC5: Invest in specification quality, not only verification power. The most sophisticated model checker applied to a semantically incorrect specification provides false assurance. Teams should invest in specification review and validation (using interactive disambiguation tools such as NL2Spec [26]) before deploying verification infrastructure. They should treat the specification as a first-class artifact subject to version control, testing, and peer review.

8.3 Threats to Validity of This Review

This review is subject to several limitations. We organize them according to four standard validity threats: internal, external, construct, and conclusion validity.

Internal validity. The review was conducted by a single reviewer, introducing the risk of subjective selection and data extraction bias. A validation check in which three raters screened a repeated 37-record search at title/abstract level (Section 3.5) gave Fleiss' $\kappa = 0.81$ and pairwise raw agreement between 86.5 % and 97.3 %, but it does not cover full-text screening or data extraction, and its two additional raters are co-authors rather than external reviewers. Borderline cases were resolved conservatively (retain if in doubt); this strategy may have inflated inclusion counts relative to a dual-reviewer protocol with explicit disagreement resolution. In the check, three of the four disagreements in which the primary reviewer stood alone were inclusions that both co-authors rejected. The same check found three corpus studies (NL2LTL, NL2Spec, NL-to-LTL/HSD) that all three raters judged out of scope and 11 in-scope records missing from the corpus. Readers should weigh the GRADE certainty ratings in Table 8 with this single-reviewer limitation in mind, since GRADE's study-limitations domain (Section 6.6) does not separately capture reviewer-count bias in the underlying corpus selection itself.

External validity. We limited the search to six English-language databases and to papers accessible in full text. The survey does not capture proprietary industry systems (e.g., OpenAI's internal safety infrastructure, Google's production agent safety layer). The primary systematic search (Section 3.3) excludes papers published after 29 May 2026; a targeted, non-systematic search (Section 3.3) identified evidence current to 30 August 2026, but it did not follow the full PRISMA protocol and its findings are reported as corroborating evidence (Sections 6.3, 7.1) rather than as formal corpus entries. The field's publication pace (multiple relevant papers per week on arXiv) means this survey represents a snapshot rather than a complete, up-to-date map.

Construct validity. The GRADE framework originated in clinical research, and we adapted it to computer science, a field its designers did not have in mind. The resulting certainty ratings reflect the reviewer's judgment of construct, internal, and external validity relative to an ideal computer science evidence standard, not a clinical one. Readers should interpret GRADE ratings in this context.

Conclusion validity. The narrative synthesis and gap analysis are theory-driven: the five thematic clusters and ten gaps were identified through the reviewer's reading of the corpus and may not capture all relevant patterns. Alternative clustering schemes (e.g., by application domain, LLM architecture, or mathematical formalism) would yield different gap structures. The research agenda in Section 7 should be read as one informed synthesis, not as a definitive enumeration of all open problems.

9 Conclusion

This survey has systematically mapped the emerging field of formal verification for LLM agent plans. Following a PRISMA 2020 protocol over six databases, we identified and synthesized 38 studies spanning specification techniques, verification approaches, enforcement mechanisms, and safety evaluation benchmarks, published between 2022 and 2026.

9.1 Summary of Findings

The synthesis in Sections 6 and 7 yields four findings and three priority gaps; each statement below points to the section that gives its evidence.

The specification bottleneck is real and underaddressed (RG1, Appendix C.1): NL-to-formal translation achieves high syntactic but low semantic correctness (Section 6.1), so verification built on it risks false assurance.

Runtime monitoring is the most mature sub-field (Section 5.5): it is the only paradigm with demonstrated production deployment, at the cost of one-step or short-horizon guarantees only.

The verifier tax is the field's most consequential single-study finding (RG3, Appendix C.3): in one τ-Bench study of transactional tool-use agents, intercepting most individually unsafe actions does not reliably produce safe task completion, with two 2025–2026 benchmarks corroborating the qualitative pattern (Section 6.3).

No current approach achieves all five desiderata simultaneously: soundness, completeness, scalability, semantic grounding, and utility preservation (Section 5.4).

9.2 Research Agenda

Of the ten gaps in Section 5.7, we consider RG1 (semantic correctness), RG3/RG7 (task-level safety and safe re-planning, jointly), and RG2 (long-horizon scalability) the highest priority, for the reasons given in their entries in Appendix C.

9.3 Limitations

A single reviewer conducted this review; a three-rater title/abstract validation check on a repeated search (Fleiss' $\kappa = 0.81$; Section 3.5) supports the screening decisions but covers neither full-text screening nor extraction, and it exposed 11 in-scope records missing from the corpus. No meta-analysis was possible due to the high heterogeneity in metrics, domains, and methods across the included studies. We limited the search to English-language, accessible papers; the review does not cover proprietary industry systems. The systematic search covers preprints through 29 May 2026; a non-systematic targeted search (Section 3.3) additionally identifies corroborating evidence through 30 August 2026, reported separately from the formal corpus. GRADE certainty is predominantly Very Low due to the novelty of the field and the absence of large, independent replications.

9.4 Closing Remarks

The case for formal verification of LLM agent plans is compelling: agents operating in consequential domains must provide guarantees that go beyond empirical performance on benchmarks. The case is also technically hard: the stochastic, NL-native nature of LLMs resists direct application of classical formal methods.

The field has made substantial progress in four years, moving from informal safety prompting to runtime LTL monitors with millisecond per-rule overhead [19], pre-execution DFA product verification over multi-framework workflow graphs [34], and probabilistic monitors that warn of violations tens of seconds before they occur [4]. But the translation bottleneck, the verifier tax, and the scalability gap remain open, and resolving them will require sustained collaboration among the formal methods, NL processing, and AI safety research communities.

The taxonomy, gap analysis, and evidence synthesis in this survey give that collaboration a shared vocabulary and a map of what the field has achieved and what remains open.

Competing interests. The authors declare no competing interests.

Acknowledgments

The authors would like to express their gratitude to the Department of Computer Science at the University of Manchester (UoM) and the Systems and Software Security (S3) Research Group for their invaluable support, collaborative environment, and access to cutting-edge resources, which were instrumental in the success of this research. We conducted this work with partial funding from the Engineering and Physical Sciences Research Council (EPSRC) grants EP/T026995/1, EP/V000497/1, EP/X037290/1, and the Soteria project, awarded by the UK Research and Innovation under the Digital Security by Design (DSbD) Programme. The funders had no role in the design, conduct, or analysis of this review, in the preparation of the manuscript, or in the decision to publish.

References

[1] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents, 2023.

[2] Lesly Miculicich, Mihir Parmar, Hamid Palangi, Krishnamurthy Dj Dvijotham, Mirko Montanari, Tomas Pfister, and Long T. Le. Veriguard: Enhancing llm agent safety via verified code generation, 2025.

[3] Adharsh Kamath, Sishen Zhang, Calvin Xu, Shubham Ugare, Gagandeep Singh, and Sasa Misailovic. Enforcing temporal constraints for llm agents, 2025.

[4] Haoyu Wang, Christopher M. Poskitt, Jiali Wei, and Jun Sun. Probguard: Probabilistic runtime monitoring for llm agent safety, 2025.

[5] Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. Agent-safetybench: Evaluating the safety of llm agents, 2024.

[6] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. Agentharm: A benchmark for measuring harmfulness of llm agents, 2024.

[7] Marcus Tantakoun, Xiaodan Zhu, and Christian Muise. Llms as planning formalizers: A survey for leveraging large language models to construct automated planning models, 2025.

[8] Keshav Ramani, Vali Tawosi, Salwa Alamir, and Daniel Borrajo. Bridging llm planning agents and formal methods: A case study in plan verification, 2025.

[9] Tanmay Sah, Vishal Srivastava, Dolly Sah, and Kayden Jordan. The verifier tax: Horizon dependent safety success tradeoffs in tool using llm agents, 2026.

[10] Gordon H. Guyatt, Andrew D. Oxman, Gunn E. Vist, Regina Kunz, Yngve Falck-Ytter, Pablo Alonso-Coello, Holger J. Schünemann, and GRADE Working Group. GRADE: An emerging consensus on rating quality of evidence and strength of recommendations. *BMJ*, 336(7650):924–926, 2008. doi: 10.1136/bmj.39489.470347.AD.

[11] Giuseppe De Giacomo and Moshe Y. Vardi. Linear temporal logic and linear dynamic logic on finite traces. In *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence (IJCAI 2013)*, pages 854–860, Beijing, China, 2013. IJCAI/AAAI Press. URL https://www.ijcai.org/Abstract/13/132.

[12] Matthew J. Page, Joanne E. McKenzie, et al. The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ*, 372:n71, 2021. doi: 10.1136/bmj.n71.

[13] Yancheng Li, Yifan Lin, Peiran Wang, Siyuan Liu, and Xin Wei. Besafe-bench: Unveiling behavioral safety risks of situated agents in functional environments, 2026.

[14] Ruoxi Chen, Yuan Sun, Jiaqi Wang, Mengdi Lv, Qi Zhang, and Yifan Zeng. Safemind: Benchmarking and mitigating safety risks in embodied llm agents, 2025.

[15] Sofia Garcia de Blas Garcia-Alcalde and Francesco Belardinelli. Atl*as: An automata-theoretic approach and tool for the verification of strategic abilities in multi-agent systems, 2025.

[16] Sajjad Khan. Verified detection and prevention of concurrency anomalies in multi-agent large language model systems, 2026.

[17] Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. Plan-bench: An extensible benchmark for evaluating large language models on planning and reasoning about change, 2022.

[18] Francesco Fuggitti and Tathagata Chakraborti. NL2LTL – a Python package for converting natural language (NL) instructions to linear temporal logic (LTL) formulas. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI-23)*, volume 37, pages 16428–16430, Washington, DC, USA, 2023. AAAI Press. doi: 10.1609/aaai.v37i13.27068.

[19] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. Agentspec: Customizable runtime enforcement for safe and reliable llm agents, 2025.

[20] Ananth Hariharan, Vardhan Dongre, Dilek Hakkani-Tür, and Gokhan Tur. Plan verification for llm-based embodied task completion agents, 2025.

[21] Eric Fang. Agentverify: Compositional formal verification of ai agent safety properties via ltl model checking. *Preprints.org*, 1(1):1–15, April 2026. doi: 10.20944/preprints202604.1029.v1.

[22] Yuntao Bai, Saurav Kadavath, et al. Constitutional ai: Harmlessness from ai feedback, 2022.

[23] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.

[24] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2022.

[25] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2022.

[26] Matthias Cosler, Christopher Hahn, Daniel Mendoza, Frederik Schmitt, and Caroline Trippel. *nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models*, pages 383–396. Springer Nature Switzerland, Cham, 2023. ISBN 9783031377037. doi: 10.1007/978-3-031-37703-7_18.

[27] Zhi Ma, Cheng Wen, Zhexin Su, Xiao Liang, Cong Tian, Shengchao Qin, and Mengfei Yang. Bridging natural language and formal specification–automated translation of software requirements to ltl via hierarchical semantics decomposition using llms, 2025.

[28] Yue Fang, Zhi Jin, Jie An, Hongshen Chen, Xiaohong Chen, and Naijun Zhan. Enhancing transformation from natural language to signal temporal logic using llms with diverse external knowledge, 2025.

[29] Yue Fang, Zhi Jin, Jie An, Hongshen Chen, Xiaohong Chen, and Naijun Zhan. Clarifystl: An interactive llm agent framework for stl transformation through requirements clarification, 2026.

[30] Michael Ahn, Anthony Brohan, et al. Do as i can, not as i say: Grounding language in robotic affordances, 2022.

[31] Zeyang Sha, Hanling Tian, Zhuoer Xu, Shiwen Cui, Changhua Meng, and Weiqiang Wang. Agent safety alignment via reinforcement learning, 2025.

[32] Christine P. Lee, David Porfirio, Xinyu Jessica Wang, Kevin Chenkai Zhao, and Bilge Mutlu. Veriplan: Integrating formal verification and llms into end-user planning. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, CHI '25, page 1–19, Yokohama, Japan, April 2025. ACM. doi: 10.1145/3706598.3714113.

[33] Jonghan Lim, Mostafa Tavakkoli Anbarani, Rômulo Meira-Góes, and Ilya Kovalenko. Logic-based verification of task allocation for llm-enabled multi-agent manufacturing systems, 2026.

[34] Melwin Xavier, Vaisakh M A, Melveena Jolly, and Midhun Xavier. Agentproof: Static verification of agent workflow graphs, 2026.

[35] Yue Chen Li, Stefan Zetzsche, and Siva Somayyajula. Dafny as verification-aware intermediate language for code generation, 2025.

[36] Debangshu Banerjee, Olivier Bouissou, and Stefan Zetzsche. Dafnypro: Llm-assisted automated verification for dafny programs, 2026.

[37] Haoxin Tu, Huan Zhao, Yahui Song, Mehtab Zafar, Ruijie Meng, and Abhik Roychoudhury. Agentic verification of software systems, 2025.

[38] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023.

[39] Danil S. Grigorev, Alexey K. Kovalev, and Aleksandr I. Panov. Verifyllm: Llm-based pre-execution task plan verification for robots, 2025.

[40] Anand Gokhale, Vaibhav Srivastava, and Francesco Bullo. Logicguard: Improving embodied llm agents through temporal logic based critics, 2025.

[41] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. Llamafirewall: An open source guardrail system for building secure ai agents, 2025.

[42] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. Understanding the planning of llm agents: A survey, 2024.

[43] Yedi Zhang, Yufan Cai, Xinyue Zuo, Xiaokun Luan, Kailong Wang, Zhe Hou, Yifan Zhang, Zhiyuan Wei, Meng Sun, Jun Sun, Jing Sun, and Jin Song Dong. The fusion of large language models and formal methods for trustworthy ai agents: A roadmap, 2024.

[44] Kun Wang, Guibin Zhang, et al. A comprehensive survey in llm(-agent) full stack safety: Data, training and deployment, 2025.

[45] Aarya Doshi, Yining Hong, Congying Xu, Eunsuk Kang, Alexandros Kapravelos, and Christian Kästner. Towards verifiably safe tool use for llm agents, 2026.

[46] Savinu Aththanayake, Chemini Mallikarachchi, Janeesha Wickramasinghe, Sajeev Kugarajah, Dulani Meedeniya, and Biswajeet Pradhan. Resqconnect: An ai-powered multi-agentic platform for human-centered and resilient disaster response. *Sustainability*, 18(2):1014, 2026. doi: 10.3390/su18021014.

[47] Ashen Sandeep, Sithum Jayarathna, Sunera Sandaruwan, Venura Samarappuli, Dulani Meedeniya, and Charith Perera. Context-aware multi-agent architecture for wildfire insights. *Sensors*, 26(3):1070, 2026. doi: 10.3390/s26031070.

[48] Max Zuo, Francisco Piedrahita Velez, Xiaochen Li, Michael L. Littman, and Stephen H. Bach. Planetarium: A rigorous benchmark for translating text to structured planning languages. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 11223–11240, Albuquerque, New Mexico, USA, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.naacl-long.560.

[49] Roham Koohestani. Agentguard: Runtime verification of ai agents, 2025.

[50] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.

[51] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 15 of *PMLR*, pages 627–635, Fort Lauderdale, FL, USA, 2011. PMLR.

[52] Howard Balshem, Mark Helfand, Holger J. Schünemann, Andrew D. Oxman, Regina Kunz, Jan Brozek, Gunn E. Vist, Yngve Falck-Ytter, Joerg Meerpohl, Susan Norris, and Gordon H. Guyatt. Grade guidelines: 3. rating the quality of evidence. *Journal of Clinical Epidemiology*, 64(4):401–406, 2011. doi: 10.1016/j.jclinepi.2010.07.015.

[53] Orna Kupferman, Moshe Y. Vardi, and Pierre Wolper. An automata-theoretic approach to branching-time model checking. *Journal of the ACM*, 47(2):312–360, 2000. doi: 10.1145/333979.333987.

[54] Rajeev Alur, Thomas A. Henzinger, and Orna Kupferman. Alternating-time temporal logic. *Journal of the ACM*, 49(5):672–713, 2002. doi: 10.1145/585265.585270.

[55] Cătălin Dima and Ferucio Laurenţiu Tiplea. Model-checking atl under imperfect information and perfect recall semantics is undecidable, 2011.

[56] Amir Pnueli. The temporal logic of programs. In *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (FOCS 1977)*, pages 46–57, Los Alamitos, CA, USA, 1977. IEEE Computer Society. doi: 10.1109/SFCS.1977.32.

[57] Edmund M. Clarke and E. Allen Emerson. Design and synthesis of synchronization skeletons using branching-time temporal logic. In *Logics of Programs, Workshop, Yorktown Heights, New York (LNCS 131)*, pages 52–71, Berlin, 1981. Springer. doi: 10.1007/BFb0025774.

[58] Oded Maler and Dejan Nickovic. Monitoring temporal properties of continuous signals. In *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems (FORMATS/FTRTFT 2004), LNCS 3253*, pages 152–166, Berlin, Heidelberg, 2004. Springer. doi: 10.1007/978-3-540-30206-3_12.

[59] Hans Hansson and Bengt Jonsson. A logic for reasoning about time and reliability. *Formal Aspects of Computing*, 6(5):512–535, 1994. doi: 10.1007/BF01211866.

[60] Edmund M. Clarke, Thomas A. Henzinger, Helmut Veith, and Roderick Bloem, editors. *Handbook of Model Checking*. Springer International Publishing, Cham, 2018. ISBN 9783319105758. doi: 10.1007/978-3-319-10575-8.

[61] Marta Kwiatkowska, Gethin Norman, and David Parker. PRISM 4.0: Verification of probabilistic real-time systems. In *Proceedings of the 23rd International Conference on Computer Aided Verification (CAV 2011), LNCS 6806*, pages 585–591, Berlin, Heidelberg, 2011. Springer. doi: 10.1007/978-3-642-22110-1_47.

[62] Alfonso Emilio Gerevini. An introduction to the planning domain definition language (pddl): Book review. *Artificial Intelligence*, 280:103221, March 2020. ISSN 0004-3702. doi: 10.1016/j.artint.2019.103221.

[63] Richard Howey, Derek Long, and Maria Fox. VAL: Automatic plan validation, continuing research on the development of language independent plan validators. In *Proceedings of the 16th IEEE International Conference on Tools with Artificial Intelligence (ICTAI 2004)*, pages 294–301, Los Alamitos, CA, USA, 2004. IEEE Computer Society. doi: 10.1109/ICTAI.2004.110.

[64] Richard E. Fikes and Nils J. Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971. doi: 10.1016/0004-3702(71)90010-5.

[65] Kutluhan Erol, James Hendler, and Dana S. Nau. Complexity results for htn planning. *Annals of Mathematics and Artificial Intelligence*, 18(1):69–93, March 1996. ISSN 1573-7470. doi: 10.1007/bf02136175.

[66] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured nlp tasks without finetuning, 2023.

[67] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS06, page 404–415, San Jose, CA, USA, October 2006. ACM. doi: 10.1145/1168857.1168907.

[68] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement for symbolic model checking. *Journal of the ACM*, 50(5):752–794, September 2003. ISSN 1557-735X. doi: 10.1145/876638.876643.

[69] Jamieson M. Cobleigh, Dimitra Giannakopoulou, and Corina S. Păsăreanu. *Learning Assumptions for Compositional Verification*, page 331–346. Springer Berlin Heidelberg, Berlin, 2003. ISBN 9783540365778. doi: 10.1007/3-540-36577-x_24.

[70] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In *Proceedings of the 35th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, POPL08, page 273–284, San Francisco, CA, USA, January 2008. ACM. doi: 10.1145/1328438.1328472.

[71] Vladimir Vovk, Alex Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer-Verlag, New York, 2005. ISBN 0387001522. doi: 10.1007/b106715.

[72] Maria Fox, Alfonso Gerevini, Derek Long, and Ivan Serina. Plan stability: replanning versus plan repair. In *Proceedings of the Sixteenth International Conference on Automated Planning and Scheduling*, ICAPS'06, pages 212–221, Cumbria, UK, 2006. AAAI Press. ISBN 9781577352709.

[73] Ivan Beschastnikh, Yuriy Brun, Sigurd Schneider, Michael Sloan, and Michael D. Ernst. Leveraging existing instrumentation to automatically infer invariant-constrained models. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*, ESEC/FSE'11, page 267–277, Szeged, Hungary, September 2011. ACM. doi: 10.1145/2025113.2025151.

[74] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, July 2009. ISSN 1557-7317. doi: 10.1145/1538788.1538814.

[75] Ozgur Dedehayir and Martin Steinert. The hype cycle model: A review and future directions. *Technological Forecasting and Social Change*, 108:28–41, July 2016. ISSN 0040-1625. doi: 10.1016/j.techfore.2016.04.005.

[76] Guy Katz, Clark Barrett, David L. Dill, Kyle Julian, and Mykel J. Kochenderfer. *Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks*, page 97–117. Springer International Publishing, Cham, 2017. ISBN 9783319633879. doi: 10.1007/978-3-319-63387-9_5.

A Background on Formal Methods and Agent Architectures

The verification techniques surveyed here rest on a small set of formal foundations: temporal logic (the language of safety properties), model checking (the decision procedure), classical planning formalisms (structured task representations), and LLM agent architectures (the target of verification). Understanding these foundations explains why temporal logic dominates the specification literature, why model checking is the most common algorithmic backbone, and why the stochastic nature of LLMs breaks classical assumptions. Section 2 gives the formal definitions that build on this material.

A.1 Temporal Logic

Temporal Logic extends classical logic with operators over sequences of states, and has become the dominant specification language in LLM-agent verification [56]. The following covers the four variants most relevant to this survey.

LTL [56] interprets formulas over infinite execution traces; for finite plans we use its finite-trace variant, LTL$_f$ [11] (Section 2). Its core grammar uses atomic propositions, negation, conjunction, and two primitive temporal operators, **X** (*next*) and **U** (*until*), from which **F** (*eventually*) and **G** (*globally*) are derived. For example, $\mathbf{G}(\neg auth \Rightarrow \neg access)$ forbids data access in any state where the agent is not authenticated, and $\mathbf{G}(tool_call \Rightarrow \mathbf{X}\, verify_output)$ mandates immediate post-call verification. LTL is the target specification language of NL2LTL [18], NL2Spec [26], Agent-C [3], AgentVerify [21], LogicGuard [40], and VerifyLLM [39].

While LTL reasons over linear execution traces, branching-time logics offer an alternative perspective by quantifying over multiple possible futures. **CTL** [57] interprets formulas over branching-time trees, pairing path quantifiers (**A**: all paths; **E**: some path) with temporal operators, enabling both reachability ($\mathbf{EF}\,\varphi$) and universal invariants ($\mathbf{AG}\,\varphi$). AgentVerify [21] exploits CTL to reason over non-deterministic LLM oracles embedded in FSMs.

Both LTL and CTL assume discrete time steps, but many agent domains require reasoning about continuous time and real-valued signals. **STL** [58] lifts LTL to continuous-time, real-valued signals with bounded temporal operators and quantitative robustness semantics: the value $\rho(\varphi, \mathbf{x}, t) \in \mathbb{R}$ measures not only whether a property holds but by how much, making it natural for cyberphysical and robotic agents [28, 29].

A further limitation of standard temporal logics is their determinism: they assume a system either satisfies a property or does not. For stochastic LLM agents, however, safety is a matter of probability. **PCTL** [59] augments CTL with probabilistic path quantifiers: $\mathbf{P}_{\geq p}[\varphi]$ asserts that φ holds with probability at least p, enabling PRISM-style model checking of stochastic systems. ProbGuard [4] relies on this to bound the probability of unsafe state visits.

A.2 Model Checking

Model checking [60] decides whether a formal model M satisfies a specification φ, written $M \models \varphi$, and underpins most automated verification approaches surveyed here.

To apply model checking, one must first represent the system in a formal modeling language. A **Kripke structure** $M = (S, S_0, R, L)$ pairs a finite state space and transition relation with a labeling function $L : S \to 2^{AP}$ that assigns atomic propositions to states. LTL and CTL formulas are then evaluated over paths or trees of paths through M. In the LLM-agent context, states capture agent configurations (task state, memory, tool-call history), transitions correspond to actions, and propositions encode safety-relevant facts such as *authenticated*, *pii_exposed*, or *budget_exceeded*. Verification systems can construct Kripke structures from ReAct-style plan traces [8] or define them a priori, with the LLM embedded as a non-deterministic oracle [21].

Given a Kripke structure and a specification, the next question is how to perform the actual verification algorithmically. **Automata-based LTL model checking** translates $\neg\varphi$ to a Büchi automaton, computes the product $M \otimes \mathcal{A}_{\neg\varphi}$, and checks for an accepting cycle (a counterexample); this underlies tools such as SPIN and NuSMV. AgentProof [34] adopts a variant that compiles safety policies to DFAs and checks compliance via a graph $\times$ DFA product over agent workflow graphs.

Deterministic automata-based methods assume that the system's behavior is fully known and predictable. When agents exhibit stochastic behavior, however, probabilistic guarantees become necessary. **Probabilistic model checking** extends this to Markov chains and MDPs, quantifying the probability of reaching an unsafe state [61]. ProbGuard [4] learns a DTMC from observed execution traces and applies PAC-learning bounds to issue probabilistic safety warnings.

A.3 Classical Planning Formalisms

Classical planning formalisms provide the structured task representations that several LLM-agent verification approaches build upon.

The most widely adopted formalism in this family is **PDDL** [62], the standard input language for AI planners. A problem comprises a *domain* (types, predicates, and action schemas with preconditions and add/delete effects) and a *problem* (objects, initial state, and goal). A valid plan is a sequence of ground actions that leads from the initial state to the goal, and tools such as VAL [63] can check it. In the LLM-agent context, LLMs are prompted to generate PDDL descriptions for an external planner [7]; the principal challenge is semantic correctness, as LLM-generated PDDL achieves high syntactic validity but only 24.8 % to 34.2 % semantic correctness on the Planetarium benchmark [48].

While PDDL remains the most common classical planning formalism, two related frameworks also appear in the verification literature. **STRIPS** [64], the predecessor to PDDL, and **HTN planning** [65], which decomposes tasks hierarchically into sub-tasks, both map naturally to the plan structures of LLM agents (e.g., ReAct, AutoGen). HTN verification is explored in the context of multi-agent task allocation [33].

A.4 LLM Agent Architectures

An *LLM agent* is a system that uses an LLM as a reasoning engine to select and sequence actions from a predefined tool set to accomplish a goal. We characterize agents along three dimensions relevant to verification.

The first dimension concerns how the agent represents its plans internally and externally, as this determines which verification methods apply. **Plan representation** takes three forms: *trace-based* (interleaved thought–action–observation

sequences [25], the most common and directly amenable to LTL evaluation), *graph-based* (predefined or dynamic control-flow graphs suited to static analysis and DFA product construction [34]), and *code-based* (executable plans verifiable via program analysis or formal contracts [2, 35]).

A second, independent dimension is the length of plans the agent must handle, as this directly affects verification tractability. **Execution horizon** distinguishes short-horizon plans (5 to 30 actions in bounded environments such as household robotics or customer service, tractable for current verifiers) from long-horizon plans (50 to 500+ actions in open-ended settings such as software engineering or autonomous web navigation, beyond the reach of existing approaches).

The third dimension captures whether a single agent acts in isolation or multiple agents interact, since verification complexity grows substantially with the number of agents. **Agency mode** ranges from single-agent systems, which produce one action sequence, to multi-agent systems, where inter-agent communication, trust delegation, and emergent interactions render per-agent verification insufficient [33].

B Relationship to Prior Taxonomies

This appendix compares the three-level taxonomy of Section 4 with three earlier classifications.

Zhang et al. [43] propose a three-way taxonomy (FM-enhancing-LLM, LLM-enhancing-FM, unified neural-symbolic) that captures the *direction of integration* between formal methods and LLMs, but does not distinguish between verification moments or pipeline stages. Huang et al. [42] classify LLM planning approaches by planning strategy (task decomposition, plan selection, external module, reflection), but do not address safety verification. A full-stack safety survey [44] categorizes safety concerns by training phase (data, training, deployment), but does not focus on formal verification.

Our taxonomy is orthogonal to these preceding frameworks: it classifies by *pipeline stage*, *verification moment*, and *formal grounding* rather than by integration direction, planning strategy, or deployment phase. Used in conjunction with prior taxonomies, it provides a more thorough characterization of any individual approach.

C Research Agenda: Full Treatment of RG1 to RG10

This appendix expands Section 7, which summarizes the directions in Table 10. For each of the ten open problems identified in Sections 5.7 and 6, we describe the current state of the art, the core technical difficulty, and two to three concrete research directions with pointers to the most relevant prior work.

C.1 RG1 – Semantic Correctness in NL-to-Formal Translation

LLM-based NL-to-formal translation achieves above 90 % syntactic correctness for LTL and above 96 % for PDDL, but only 24.8 % to 34.2 % semantic correctness for PDDL on Planetarium [48]. LTL translation accuracy under interactive disambiguation reaches 86.11 %, and 44.4 % without it [26]. These figures cannot be compared with the PDDL figure, because NL2Spec's benchmark and evaluation protocol differ from Planetarium's. Interactive disambiguation (NL2Spec [26], ClarifySTL [29]) and hierarchical decomposition [27] reduce semantic errors but do not eliminate them. Current evaluations assess semantic correctness after generation, using equivalence checks against ground-truth specifications or model checking against ground-truth traces; neither kind of ground truth exists for novel domains.

Checking semantic correctness requires a formal model that correctly represents the intended domain semantics. For novel domains, however, we lack any independent ground truth apart from the specification we are evaluating. The problem is therefore circular: to verify that a specification is semantically correct, one needs a formal oracle that is itself a correct formalization of the domain.

RG1.1 Grammar-constrained decoding for formal languages: LLM decoding can be constrained to produce only syntactically valid outputs by restricting the token distribution at each step to tokens consistent with the formal grammar [66]. Extended to semantic constraints (e.g., well-typedness of predicate arguments in PDDL), constrained decoding can eliminate a class of semantic errors at generation time. Integrating semantic type systems for LTL and PDDL into decoding constraints is an open engineering challenge.

RG1.2 Equivalence checking via counter-example guided synthesis: Given two candidate translations of the same NL requirement, a model checker can determine whether they are semantically equivalent by searching for a Kripke structure that satisfies one but not the other. If the system finds a counterexample, it can return the

discrepancy to the LLM as feedback for refinement, in a CEGIS [67] loop adapted to specification translation. NL2Spec's disambiguation loop [26] is a step in this direction; full CEGIS integration remains unstudied.

RG1.3 Semantic ground-truth benchmarks: Progress on semantic correctness requires benchmark datasets that pair each NL requirement with a formally verified LTL/PDDL specification and a corpus of execution traces that definitively test semantic correctness. Such benchmarks do not yet exist at scale for agent safety domains. Crowdsourcing requirements from real agent deployment logs, verified by domain experts, is a feasible near-term path.

C.2 RG2 – Scalability to Long-Horizon Plans

Current verified systems handle plans of 5 to 30 actions [3, 19, 39]. AgentProof [34] scales to graphs of 5,000 nodes in pre-deployment static verification, but this applies to finite workflow graphs, not open-ended dynamic plans. Long-horizon tasks (SWE-bench: 100+ actions; autonomous research assistants: 500+ actions) are beyond the reach of exhaustive model checking. The number of states in a Kripke structure grows exponentially with plan length. For open-ended LLM agents, the branching factor at each step is the model's vocabulary size ($\sim$100k tokens), making exhaustive model checking computationally infeasible.

Section 7.1 analyzes how this cost compounds with the number of agents (Table 11).

RG2.1 – Partial verification of safety-critical sub-sequences. Many long-horizon plans contain a small number of safety-critical segments (e.g., authentication sequences, file deletion chains, financial transaction sequences) surrounded by safe filler actions. Identifying these sub-sequences automatically, with static analysis tagging high-risk action types, and verifying only them could provide partial verification guarantees at manageable cost. The AgentVerify compositional framework [21] provides a foundation; targeted identification of safety-critical sub-sequences within it is unstudied.

RG2.2 – Abstraction-refinement for agent plans. Counterexample-guided Abstraction Refinement (CEGAR) [68] starts with a coarse abstraction of the state space, model-checks the abstraction, and refines it when it finds a spurious counterexample. When adapted to LLM agent plans, CEGAR could abstract away low-risk action types (e.g., read-only queries) and focus verification effort on high-risk transitions, refining the abstraction whenever a counterexample turns out to be spurious.

RG2.3 – Compositional verification with assume-guarantee contracts. Long-horizon plans decompose into segments with formal interface contracts: each segment assumes a set of preconditions and guarantees a set of postconditions [69]. Verifying each segment against its contract and composing the proofs avoids the need to model-check the full plan trace. AgentProof [34] demonstrates segment-level verification; assume-guarantee composition across dynamically generated segments remains open.

C.3 RG3 – The Verifier Tax

One τ-Bench study of transactional tool-use agents shows that even when a system intercepts 94 % of individually unsafe actions, SSR remains below 5 %, driven by integrity leaks: agents hallucinating credentials to bypass blocked paths [9]. No existing system resolves this gap. Agent-C achieves 100 % action conformance but does not report SSR [3]; AgentSpec reports task success rates but not SSR as defined in the literature [9, 19]. The verifier tax is a goal-optimization phenomenon: when the direct path to the goal is blocked, the agent searches for an alternative path, which may be unsafe in ways the monitor does not anticipate. Blocking actions one by one without considering the agent's goal makes the enforcement gameable.

RG3.1 – Plan-level safety policies. Instead of checking individual actions, a plan-level safety monitor checks a proposed sub-sequence or full plan fragment against properties that reference the goal context. This requirement means the monitor must have access to the agent's goal representation (e.g., the task description and current intent) and the action sequence. Agent-C's temporal DSL [3] is a step toward this; extending it to goal-contextual properties is an open problem.

RG3.2 – Credential and identity integrity constraints. In the integrity-leak mechanism, the agent fabricates identifiers that appear plausible but do not exist in the ground-truth user database. A dedicated integrity monitor would close this attack vector: it maintains a ground-truth index of valid identifiers and rejects any action that references an identifier outside the index. This mechanism parallels SQL injection prevention via parameterized queries, in which the system rejects inputs that deviate from a trusted schema.

RG3.3 – Safe replanning upon violation. Rather than simply blocking a violating action, the enforcement system should trigger a replanning request: generate an alternative action sequence from the current state that

satisfies all safety constraints and makes progress toward the goal. This replanning requirement integrates the enforcement monitor with a constrained planner (e.g., a PDDL solver or an LLM with hard constraints), and this integration represents the most technically ambitious direction for resolving the verifier tax. VeriPlan [32] demonstrates model-checker-guided user-facing replanning; adapting this to automated agent replanning is an open problem.

C.4 RG4 – Dynamic Plan Generation

Static verification (AgentProof [34], VeriGuard offline [2]) requires pre-specified finite workflow graphs. Most real LLM agent deployments generate plans dynamically in response to environment state: the agent observes the current state, generates the next action, executes it, observes the outcome, and repeats. No included study provides pre-execution soundness guarantees for dynamically assembled plans.

RG4.1 – Online workflow graph extraction. As an agent executes, the sequence of calls it makes to sub-agents and tools implicitly defines a workflow graph. Extracting this graph in real time and running incremental static checks (adding each new node and edge as the system encounters them) could provide online structural verification without requiring the full graph to be pre-specified. AgentProof's framework [34] is a candidate starting point; online incremental model checking over dynamically growing graphs is an active area in formal methods.

RG4.2 – Verified plan template instantiation. Many agent tasks share high-level plan structures (templates) that differ only in parameter instantiation. Pre-verifying these templates against safety properties, and then checking that each dynamically generated instantiation conforms to its verified template, would provide soundness guarantees for parameterized plan families. This verification strategy parallels type-checked generics: the system applies the type system (verifier) to the template, and instantiations inherit the proof.

C.5 RG5 – Multi-Agent Verification

Most verification work targets single-agent plans. Discrete-event-system LTL verification has been applied to multi-robot task allocation in manufacturing [33], and AgentVerify addresses MCP and skill-invocation protocols in multi-agent settings [21]. However, no study provides a general compositional verification framework for arbitrary multi-agent system topologies. Multi-agent systems exhibit emergent interaction patterns, such as role confusion, trust delegation, cascade failures, and collusion, that are not reducible to the properties of individual agents. Verifying a property of a multi-agent system requires reasoning about the joint execution of all agents, which compounds the state-space explosion problem.

Section 7.1 shows that composing agents changes the complexity class of the verification problem, audits the two multi-agent corpus studies against that result, and tabulates verification cost against plan length and agent count (Table 11).

RG5.1 – Contract-based interfaces between agents. We can specify each agent using an assume-guarantee contract: a formal interface that describes the inputs it assumes and the outputs it guarantees [69]. Composing these contracts allows us to verify system-level properties from component-level proofs without exhaustive joint state-space exploration. This approach is standard in concurrent system verification; adapting it to LLM agents with non-deterministic outputs presents an open challenge.

RG5.2 – Protocol verification for inter-agent communication. In multi-agent systems, agents communicate via structured message-passing protocols. Formalizing these protocols as session types or communicating automata [70] and checking them for safety properties (no deadlock, no privilege escalation) is a tractable sub-problem of multi-agent verification. The MCP/skill invocation protocols of AgentVerify [21] are a starting point.

C.6 RG6 – Probabilistic Specification and Stochastic Model Checking

LLMs are inherently stochastic: the same plan generation call with the same prompt can produce different action sequences across runs. Therefore, we should express safety properties for stochastic agents probabilistically ("the agent violates the policy with probability less than ϵ") rather than deterministically. ProbGuard [4] is the only included study that addresses probabilistic safety; it uses DTMC models with PAC-learning bounds. Outside the corpus, AgentGuard [49] learns an MDP of agent behavior online and applies probabilistic model checking to it at runtime; no included study applies PRISM-style PCTL model checking to LLM agent plans.

RG6.1 – PCTL specifications for LLM agent safety properties. We can lift standard safety specifications (e.g., "authenticate before data access") to probabilistic versions: "the probability of accessing data without prior authentication is less than 0.01". PCTL provides the formal language for such statements; PRISM provides a tool to check them against DTMC or MDP models. Adapting PRISM to LLM agent execution traces, by learning the DTMC from empirical traces and model-checking PCTL properties against it, is a feasible and direct extension of ProbGuard.

RG6.2 – Calibrated uncertainty quantification for safety monitors. Current deterministic monitors make binary block/allow decisions. A monitor that quantifies its own uncertainty, outputting a probability of violation rather than a binary verdict, can be integrated with risk-threshold policies, enabling deployments to explicitly calibrate the safety-utility trade-off. Conformal prediction [71] provides a distribution-free framework for calibrated uncertainty quantification, but researchers have not yet applied it to agent safety monitoring.

C.7 RG7 – Verified Re-Planning

When a plan violates a safety property, all included enforcement systems except Agent-C either block the action, log and continue, or abort the task; Agent-C substitutes a compliant alternative for the single violating tool call [3]. None generates an alternative plan segment that satisfies all safety constraints and continues toward the goal. VeriPlan [32] generates user-facing constraint-satisfying plans in end-user planning settings but does not address automated replanning for agentic systems.

RG7.1 – Constraint-satisfying plan repair. Plan repair [72] modifies a failing plan minimally to restore validity. Integrated with an LTL or PDDL constraint checker, plan repair could generate an alternative plan fragment from the point of violation that satisfies all checked properties. The challenge for LLM-based agents is that plan repair operates on formal plan representations, whereas LLM plans are in NL, so the two must be linked by a bidirectional translation pipeline.

RG7.2 – Constrained LLM plan generation. Rather than repairing a violating plan post-hoc, the enforcement system can impose hard constraints on the LLM's generation process, steering the model toward safe plan completions using constrained beam search or SMT-guided decoding. Agent-C's SMT-based constrained decoding [3] demonstrates feasibility for temporal sequence constraints; extension to arbitrary LTL properties over plan prefixes is an open problem.

C.8 RG8 – Specification Acquisition at Scale

Writing formal safety specifications (LTL formulas, AgentSpec DSL rules, PDDL constraints) requires formal methods expertise. This expertise requirement creates a scalability bottleneck: there are far more agent-deployment domains than there are formal-methods experts. Existing approaches to automated specification acquisition, such as pattern-based translation of NL instructions [18] and interactive disambiguation of NL requirements [26], remain nascent and have seen evaluation only in narrow domains.

RG8.1 – Specification mining from demonstration traces. Given a corpus of labeled agent execution traces (safe vs. unsafe), specification mining algorithms [73] can infer LTL formulas that distinguish safe from unsafe traces. Applying this to LLM agent execution logs, which production deployments increasingly produce, would enable data-driven specification acquisition without human expert authoring.

RG8.2 – Specification extraction from regulatory and policy documents. Many agent deployment domains fall under formal regulations, such as GDPR, HIPAA, and financial compliance rules. Extracting formal safety specifications from these documents, with LLMs as translators under expert validation, would reuse existing domain knowledge and produce reusable specification libraries. The literature demonstrates the principle for unstructured text; no study has yet applied it at scale to structured regulatory documents [26].

RG8.3 – Compositional specification libraries. A shared, peer-reviewed library of reusable LTL and DSL specifications for common agent safety properties (authentication-before-access, data-minimization, human-gate-before-irreversible-action) would reduce the per-deployment specification burden. AgentVerify's four-domain specification library [21] provides a seed, but the community needs to curate it systematically.

C.9 RG9 – Adversarial Robustness of Verification Systems

Enforcement systems themselves present attack surfaces. Production-grade guardrails still allow 1.75 % attack success [41]. Verification components that rely on LLM-based translation are additionally vulnerable to adversarial inputs

that cause the LLM to generate incorrect specifications or Kripke structures [44]. Adversarial evaluation of verification and enforcement systems is systematically absent from the literature.

RG9.1 – Red-teaming of verification pipelines. Characterizing the attack surface requires systematic adversarial evaluation of the end-to-end pipeline, from NL requirement elicitation through specification translation and model checking to enforcement. Adapting LLM red-teaming methodologies [6] to target the verification pipeline specifically (prompt injection into specification generators, adversarial Kripke structures, monitor bypass via indirect action sequences) would characterize the residual risk of each component.

RG9.2 – Formal robustness guarantees for specification translators. If the specification translator is itself formally verified (e.g., through type-safe grammars or certified compilation), its correctness guarantee is robust to adversarial inputs by construction. Applying verified compilation techniques [74] to LTL-generating components, at least for a restricted fragment of the input language, would provide robustness without relying on empirical red-teaming.

C.10 RG10 – Evaluation Standardization

No benchmark simultaneously evaluates formal verification coverage, runtime overhead, and SSR across multiple agent architectures and task domains. Existing benchmarks address subsets: Agent-SafetyBench [5] covers safety but not formal verification; τ-Bench [50] enables SSR measurement but only in a narrow transactional domain; AgentBench [1] covers task success but not safety. The Verifier Tax paper [9] introduced SSR as a unified metric but evaluated it on a single benchmark.

RG10.1 – A unified formal safety evaluation suite. A benchmark should define: (a) a set of task environments with known ground-truth safe and unsafe plans; (b) a corresponding library of formal safety specifications for each task; (c) standardized metrics including SSR, formal verification coverage (a formal language captures only a fraction of possible safety properties), and runtime overhead. The formal methods, NLP, and AI safety communities should build this benchmark jointly so that it covers all relevant dimensions. As an illustrative and explicitly uncalibrated headline metric, reported alongside the three-axis vector of Section 6.7 rather than in its place, we propose $\text{ESI} = \text{SSR} \cdot \min\big(1, (L_{\text{budget}}/L)^{\alpha}\big)$ for a deployment-chosen exponent α, or more simply $\text{SSR}@L_{\text{budget}}$ (SSR counting only tasks completed within the latency budget). Every included study that reports runtime overhead uses a different task domain, latency budget, and definition of acceptable degradation (Table 5), so neither formula can be fit or validated against the current corpus; we present both as candidates for the field to calibrate and test, not as validated metrics.

RG10.2 – Adoption of SSR as the primary metric. The field should standardize on SSR [9] as the primary evaluation metric for enforcement systems, supplemented by action-level conformance as a secondary metric. SSR captures the end-to-end objective (tasks completed both safely and correctly) and avoids the false confidence that action-level compliance rates alone provide. A community-maintained leaderboard or shared evaluation server would accelerate metric adoption.

RG10.3 – Benchmark contamination mitigation. LLMs trained on internet data may have memorized evaluation scenarios from public benchmarks, inflating reported performance. Mitigation strategies include held-out evaluation sets updated after each major model release, procedurally generated tasks with verifiable ground truth, and canary task detection for known contamination in benchmarks.

D Extended Discussion

This appendix extends the discussion in Section 8.

D.1 Field Maturity Assessment

The field of formal verification for LLM agent plans is at an early but structurally sound stage of development. Using the Gartner Hype Cycle [75] as a rough heuristic, the field appears to be transitioning from the *Peak of Inflated Expectations* (2023 to 2024, marked by many demonstration papers with limited evaluation) toward the *Slope of Enlightenment* (2025–2026, with the first empirical findings that challenge simplistic assumptions, such as the verifier tax [9]).

Indicators of early **maturity** include convergence on a dominant specification language (LTL) with consistent findings across independent groups [3, 18, 19, 26]; the emergence of dedicated workshops at major software engineering and programming-languages venues as forums for this research; production deployment of at least one system (LlamaFirewall [41] at Meta) with disclosed performance metrics; the development of standardized benchmarks

(Agent-SafetyBench [5], τ-Bench [50]) that enable cross-system comparison; and the first negative empirical result that challenges field assumptions (verifier tax [9]).

Indicators of **immaturity** include that GRADE certainty is *Very Low* for seven of the nine principal claims, with none reaching *High*; most studies are single-group, single-domain evaluations without independent replication; no system simultaneously satisfies soundness, completeness, scalability, semantic grounding, and utility preservation; the field lacks a shared formal framework, since every paper introduces its own agent model, specification language fragment, and evaluation protocol; and specification acquisition and verified re-planning (arguably the most important problems) are the least studied.

The maturation trajectory of this field parallels the early development of neural network verification (2017 to 2020), which similarly began with demonstrations on narrow benchmarks (MNIST classifiers, small ReLU networks) before confronting scalability and semantic gap challenges that required fundamental theoretical advances [76]. Neural network verification is now a mature subfield with verified industrial applications (e.g., the ACAS Xu collision-avoidance networks for unmanned aircraft [76]). The LLM agent verification field is approximately three to four years behind this trajectory, with the additional complication that the *output* of LLMs is NL rather than a fixed-dimension vector, which makes the semantic gap structurally harder to close.

D.2 Relationship Between Safety Paradigms

The included studies reflect three intellectual traditions for addressing LLM agent safety, and these traditions rarely engage with one another.

The **formal methods tradition** (AgentVerify [21], AgentProof [34], AutoRocq [37], VeriGuard [2]) draws on 50+ years of model checking and theorem proving research. This tradition prioritizes soundness and completeness: if verification says a plan is safe, it *is* safe. The limitation is that soundness guarantees depend on the correctness of the specification and the model, and LLMs with non-negligible semantic error rates construct both.

The **Machine Learning (ML) safety/alignment tradition** (Constitutional AI [22], Agent Safety Alignment via RL [31], Self-Refine [38]) draws on reinforcement learning, RLHF, and iterative refinement. This tradition prioritizes scalability and generality: trained models implicitly apply safety constraints to arbitrary inputs without per-deployment specification engineering. The limitation is the absence of formal guarantees: a model trained to be safe can generate unsafe outputs at inference time, and there is no formal procedure to confirm that it cannot.

The **security engineering tradition** (LlamaFirewall [41]) draws on systems security, threat modeling, and defensive programming. This tradition prioritizes adversarial robustness and production deployability: the threat model is an active adversary attempting to exploit the agent, not a passive environment. The limitation is that this tradition is primarily empirical (a reduction in attack success rate) rather than formal (a provable bound on attack success rate).

The **need for integration** follows directly. Each tradition covers a different part of the safety landscape and has different failure modes. Deployed systems will need to combine elements of all three: formal runtime monitors (providing sound action-level guarantees) over aligned base models (reducing the frequency of violations that reach the monitor) within hardened system architectures (defending against adversarial exploitation of the monitoring infrastructure). None of the included systems has yet demonstrated this integration.

D.3 Societal and Ethical Considerations

The deployment of formal verification for LLM agents raises societal and ethical questions that extend beyond technical correctness. Three concerns merit particular attention: the pressure to deploy unsafe systems, the politics of specification authorship, and the risk of over-reliance on formal certificates.

The deployment pressure paradox. The pace of LLM agent deployment in production settings far exceeds the pace of formal safety verification's maturation. Agents are being deployed in healthcare, legal, and financial settings, while GRADE certainty for seven of the nine principal claims remains *Very Low*. The societal consequence of this gap is that the harm reduction that current verification systems achieve, while real, is not yet formally guaranteed or independently replicated at scale. Survey findings should inform deployment governance: requiring SSR measurement, mandatory specification review, and periodic re-evaluation against updated benchmarks as agent capabilities evolve.

Specification as a power structure. Who writes the formal safety specification for an LLM agent determines which behaviors are permitted and which are blocked. Specification acquisition at scale (RG8) raises governance questions: specifications derived from corporate policy documents may protect business interests at the expense of user interests. In contrast, those derived from regulatory documents may lag behind technological capabilities. Transparent, publicly

accessible specification libraries with explicit authorship and versioning would reduce the risk of opaque specification capture.

The completeness illusion. A formal safety certificate declares: "This agent has been verified against specification φ", and creates a social expectation of safety that may not be warranted when φ fails to capture all relevant safety properties. Formal verification of LLM agent plans should present a partial safety guarantee conditioned on specification completeness, not a blanket safety assurance.